



Proiect de diplomă

Coordonator științific: Obreja Marius-Emanuel
Grad didactic titlu științific: Asist.univ.drd.ing.

Absolvent:
Bîrsan Lucian

Folosirea metodelor de Machine Learning pentru clasificarea stadiilor retinopatiei diabetice

Coordonator științific, Obreja Marius-Emanuel

Grad didactic titlu științific: Asist.univ.drd.ing.

Absolvent,

Bîrsan Lucian

Cuprins

Capitolul 1.....	6
1.1 Introducere.....	6
1.1.1 Problema	6
1.1.2 Sfera cercetării	6
Capitolul 2.....	7
2.1 Obiective.....	7
2.1.1 Obiectivele proiectului	7
2.1.2 Obiective algoritmice și de Machine Learning	7
2.1.3 Obiective de integrare software și interfață:	8
Capitolul 3.....	9
3.1 Metodologia și abordarea	9
3.1.1 Retinopatia diabetică	9
3.1.2 Metode de Machine Learning in domeniul medicinei	10
3.1.3 Rețele Neuronale Convoluționale	11
3.1.4 Straturi Convoluționale	11
3.1.5 Pooling Layers	12
3.1.6 Dense Layers.....	12
3.1.7 Tranziția de la arhitecturi CNN tradiționale la arhitecturi moderne	12
3.1.8 Provocări in antrenarea rețelelor CNN	13
3.1.8 Hiperparametrii și optimizatorul.....	13
3.1.9 Overfitting, Underfitting și Dropout.....	15
3.1.10 Augmentarea Datelor	16
3.1.11 Problema “Vanishing gradient”	16
3.1.12 Transfer Learning	18
3.1.13 Lucrari inrudite	18
3.1.14 Resursele pentru elaborarea proiectului	20

3.2 Implementare	22
3.2.1 Hardware și software	22
3.2.2 Nvidia Jetson Nano Developer Kit	23
3.2.3 Analiza interfețelor de conectivitate și periferice pentru JetsonNano	24
3.2.3 Setul de date	25
3.2.4 Arhitectura modelului bazat pe Transfer Learning și prevenirea supraantrenării (Overfitting)	26
3.2.5 Regularizarea Rețelei Neuronale Convoluționale	28
3.2.6 Învațarea prin transfer (Transfer-Learning)	30
3.2.7 Platforma de Virtualizare: Docker	31
3.2.8 Principiul de Funcționare al Containerelor	31
3.2.9 Argumente Privind Popularitatea și Utilizarea Docker	32
3.2.10 Docker pentru proiectul meu	32
Capitolul 4	34
4.1 Rezultate	34
4.2 Evoluția pierderii (loss)	36
4.3 Optimizarea pentru un mic boost de performanță	37
4.4 Tranziția de la Preprocesarea Standard la o accentuare a detaliilor	38
4.5 Reducerea dimensiunii de batch (batch size)	38
4.6 Ajustarea a ratei de dropout de la 0.4 la 0.6	39
4.7 Identificarea punctului optim pentru rata de învățare	40
4.8 Testarea directă pe placuța Jetson Nano	41
4.9 Rezultatele testelor direct pe plăcuța Jetson Nano	42
4.10 Dezavantajele proiectului	45
Capitolul 5	47
5.1 Posibile optimizări și direcții viitoare de dezvoltare	47
5.2 Concluzii	48
Bibliografie	50
Anexa	52

LISTA FIGURILOR

Figure 1 Evolutia retinopatiei diabetice in functie de stadiu [1]	7
Figure 2 Stagiile de diabet zaharat [2]	9
Figure 3 Exemplu de strat convolutional [3]	11
Figure 4 Exemplu de pooling layer [4]	12
Figure 5 Exemplu de CNN [5]	13
Figure 6 Exemplu de learning rate [6]	15
Figure 7 Exemplu de overfitting si underfitting [7]	15
Figure 8 Exemplu de augmentare de date [8]	16
Figure 9 Apariția problemei de vanishing gradient [9]	17
Figure 10 Exemplu de inceptie modul [10]	20
Figure 11 Un sketch ce arată pașii pentru crearea aplicației [11]	21
Figure 12 Distribuția setului de date [12]	22
Figure 13 Placa Nvidia Jetson Nano Developer Kit [13]	24
Figure 14 Sparse Categorical Crossentropy [14]	27
Figure 15 Overfitting intr-un model: test vs train plot [15]	28
Figure 16 Exemplu de dropout [16]	29
Figure 17 Funcționarea containerelor [17]	32
Figure 18 Apariția Vanishing gradient [18]	35
Figure 19 Matricea confuziilor [19]	36
Figure 20 Graficul pentru loss [20]	37
Figure 21 Raport cu rezultatele clasificărilor [21]	37
Figure 22 Graph-ul pentru rezultatele de test + train dupa optimizare [22]	39
Figure 23 Graph-ul pentru rezultatele de loss [23]	40
Figure 24 Raport cu rezultatele clasificărilor optimizate [24]	40
Figure 25 Matricea confuziilor pentru modelul optimizat [25]	41
Figure 26 Rezultatul NoDr din aplicatie direct de pe placa JetsonNano [26]	42
Figure 27 Rezultatul Severe din aplicație direct de pe placa Jetson Nano [27]	43
Figure 28 Rezultatul Proliferative din aplicație direct de pe placa Jetson Nano [28]	44
Figure 29 Rezultatul Moderate direct din aplicația Jetson Nano [29]	45

Capitolul 1

1.1 Introducere

1.1.1 Problema

Retinopatia diabetică, cunoscută pe scară largă sub numele de boală oculară diabetică, este o afecțiune medicală cauzată de tulburarea medicală a diabetului. Diabetul este o boală moderat de comună care afectează aproximativ 425 de milioane de oameni în 2017. În plus, estimările furnizate de Karadeniz (2017) arată o creștere alarmantă a cazurilor, cu un număr de peste 600 de milioane de persoane estimate să sufere de diabet până în anul 2045. Mai mult, statisticile lor demonstrează că 1 din 2 persoane nici măcar nu sunt conștiente că au această afecțiune.

Retinopatia diabetică apare de obicei ca o complicație la pacienții care suferă de diabet de mai mult de 10 ani. De când a fost descoperită insulina, în anul 1921, ambele tipuri de diabet, tipul 1 și tipul 2, au încetat să mai fie letale, deoarece oamenii au început să trăiască timp de decenii atunci când primesc tratament adecvat (Karadeniz, 2017). În consecință, numărul cazurilor de retinopatie diabetică a crescut și continuă să crească, deoarece afecțiunea a devenit una dintre principalele cauze de orbire la nivel global.

1.1.2 Sfera cercetării

Setul de date utilizat pentru antrenarea și testarea modelelor de învățare automată este disponibil public pe platforma Kaggle (Warner, 2019). Acesta constă în imagini ale fundului de ochi provenite din competiția *APTOS 2019 Blindness Detection*, împreună cu etichetele corespunzătoare care indică stadiile de severitate ale bolii, așa cum a fost menționat anterior. Clasificarea gradului de severitate al imaginilor a fost realizată de către medici oftalmologi și clinicieni certificați. Pentru a asigura uniformitatea și a optimiza procesul de antrenare, toate imaginile din setul de date au fost redimensionate la o rezoluție de 1024x1024 pixeli. Deoarece imaginile au fost preluate cu dispozitive fotografice diferite, acestea pot prezenta variații în ceea ce privește luminozitatea și claritatea.

Capitolul 2

2.1 Obiective

2.1.1 Obiectivele proiectului

Scopul proiectului a fost dezvoltarea unui model de învățare automată (*machine learning*) bazat pe rețele neuronale convoluționale care să prezică cu acuratețe dacă retinele sunt expuse la retinopatie diabetică și, în caz afirmativ, cât de avansat este stadiul afecțiunii. Pe baza rezultatului, pacienții ar trebui să poată să își trateze boala în mod corespunzător, fapt care ar trebui să reducă procentul persoanelor care își pierd vederea ca urmare a acestei complicații. Mai mult, instrumentul urmărește să funcționeze ca un indicator de sănătate mai ieftin și mai accesibil, precum și ca o alternativă eficientă de suport și metodă de îndrumare pentru oftalmologi.

Programul va prelua ca intrare o imagine a fundului de ochi și va afișa stadiul de severitate, corelat cu clasificarea clinică standard ilustrată în Figure 1. Astfel, gradele de evoluție ale afecțiunii sunt reprezentate prin următoarele clase:

-
- 0 – Fără retinopatie diabetică
- 1 – Pericol ușor de retinopatie diabetică
- 2 – Stadiu moderat de retinopatie diabetică
- 3 – Stadiu sever de retinopatie diabetică
- 4 – Retinopatie diabetică proliferativă

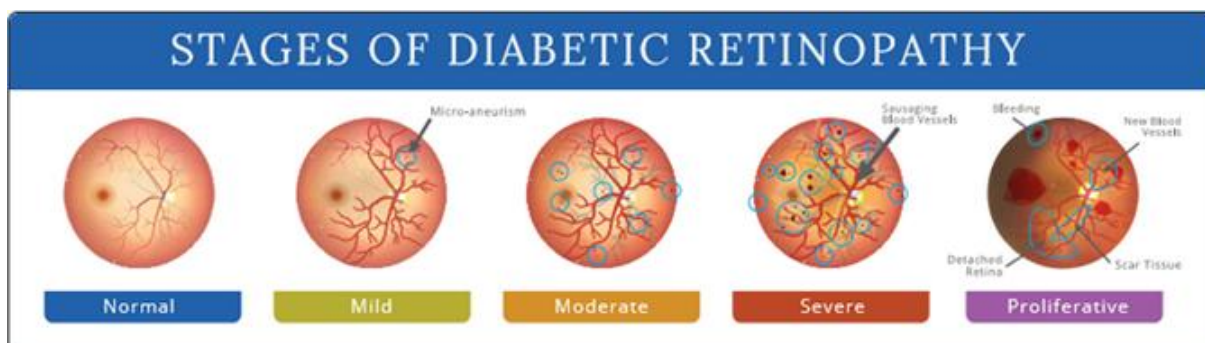


Figure 1 Evoluția retinopatiei diabetice în funcție de stadiu [1]

2.1.2 Obiective algoritmice și de Machine Learning

Pentru a atinge acest obiectiv, am definit si indeplinit urmatoarele obiective specifice ale proiectului:

1. Cercetarea si selectarea unei arhitecturi: Am ales EfficientNetB0 deoarece este optimizat pentru sistemele embedded, avand un număr redus de parametrii si acuratețe ridicata.
2. Implementarea tehnicilor de Transfer-Learning si Fine-Tuning: Aceste tehnici au fost folosit pentru a adapta rețeaua la un set de date specific imaginii fundului de ochi, folosind o rezolutie de intrare de 320x320 pixeli.

2.1.3 Obiective de integrare software și interfață:

1. Dezvoltarea unei aplicații de procesare video in timp real: Pentru atingerea acestui obiectiv s-a folosit librăria OpenCV care este capabila să preia fluxul de imagini de la o cameră digitală.
2. Crearea unei interfețe grafice: S-a creat o interfata grafica care să afișeze la fiecare refresh clasa diagnosticată și timpul de raspuns in milisecunde.

Capitolul 3

Secțiunea următoare își propune să ofere cititorului informațiile esențiale necesare pe parcursul parcurgerii lucrării. Principalele puncte abordate vizează problema pe care proiectul o soluționează, metodele utilizate pe parcursul implementării, precum și abordările anterioare ale temei respective.

3.1 Metodologia și abordarea

3.1.1 Retinopatia diabetică

[19] Așa cum a fost menționat în secțiunea de Introducere, retinopatia diabetică este o afecțiune oculară care apare în principal la persoanele care suferă de diabet zaharat. Având în vedere creșterea constantă a numărului de persoane diagnosticate cu diabet, măsurile concepute pentru prevenirea complicațiilor cauzate de retinopatia diabetică sunt absolut necesare. Simptomele cel mai des întâlnite la persoanele suspecte de această boală includ: „pete sau corpuri plutitoare în câmpul vizual”, „vedere încețoșată”, „prezența unei zone goale, întunecate, în centrul vederii” și „dificultăți de vedere” (American Optometric Association, 2020).

[29] În ceea ce privește stadiile de severitate ale bolii, acestea pot fi clasificate în două categorii generale, care, la rândul lor, sunt formate din alte cinci stadii intermediare mai precise. Într-o lucrare de Wilkinson et al. (2003), aceste cinci stadii detaliate au fost împărțite în trei stadii cu risc scăzut, un stadiu de retinopatie diabetică non-proliferativă severă și un stadiu final de retinopatie diabetică proliferativă, evoluție clinică ilustrată în Figure 2.

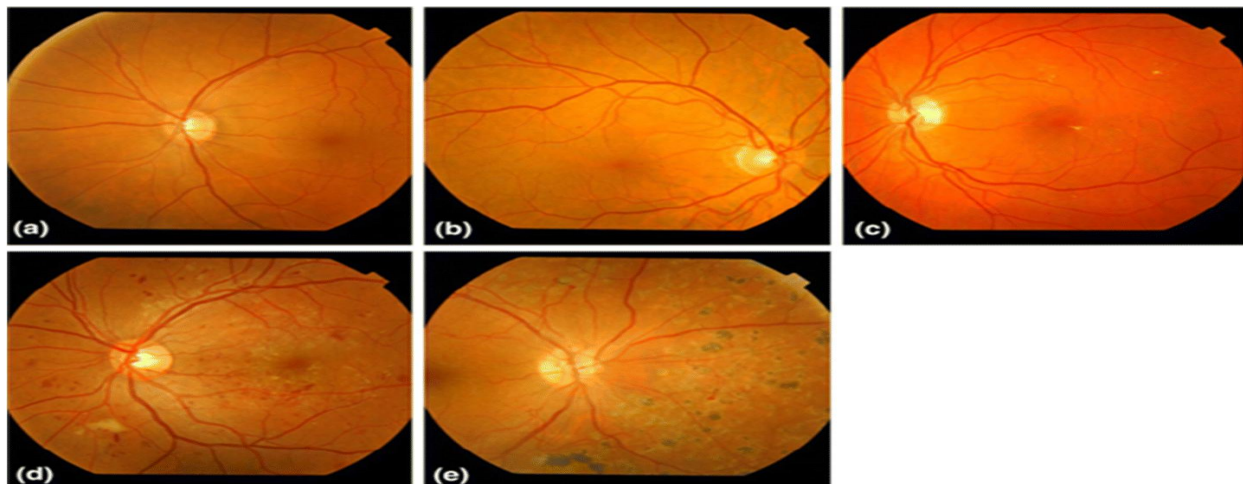


Figure 2 Stagiile de diabet zaharat [2]

Cele cinci stadii sunt determinate de numărul de microanevrisme, hemoragii retiniene, exsudate dure sau alte anomalii care pot fi observate în imaginea fundului de ochi.

Primul stadiu, denumit „Fără RD” (Retinopatie Diabetică), este atribuit unei imagini în care nu se observă anomalii. Al doilea stadiu, cunoscut sub numele de RD non-proliferativă ușoară, este prezent atunci când în imagine apare cel puțin un microanevrism. Al treilea stadiu este RD non-proliferativă moderată, unde se pot observa mai multe microanevrisme, hemoragii retiniene sau exsudate dure. Cel de-al patrulea stadiu, numit RD non-proliferativă severă, este prezent atunci când apar numeroase microanevrisme și hemoragii retiniene în toate cele patru cadrane ale imaginii retiniene. Ultimul stadiu este cunoscut sub numele de RD proliferativă, unde se observă prezența neovascularizației (formarea de noi vase de sânge).

3.1.2 Metode de Machine Learning în domeniul medicinei

În ciuda faptului că învățarea automată (*Machine Learning*) este percepută ca fiind un domeniu relativ nou, conceptul există încă de la începutul anilor '50. Persoana care a introdus această noțiune a fost Arthur Samuel, dezvoltator IBM la acea vreme, care anterior inițiasse algoritmi de tăiere alpha-beta și minimax (Foote, 2019).

Proiectul este implementat folosind metode de învățare profundă (*deep learning*). Învățarea profundă reprezintă un subset al învățării automate în care rețelele neuronale profunde sunt organizate în straturi de „neuroni” și au ca scop funcționarea într-un mod similar creierului uman.

Într-o lucrare care se concentrează pe conceptul de învățare profundă, LeCun et al. (2015) au reușit să îl explice în detaliu, afirmând totodată că acesta a adus îmbunătățiri valoroase în domenii precum recunoașterea vizuală a obiectelor, detecția obiectelor și procesarea imaginilor și a videoclipurilor. Tipul de rețele neuronale artificiale care performează cel mai bine în cazul problemelor de acest gen sunt rețelele neuronale convoluționale profunde.

Miotto et al. (2018) au subliniat faptul că, odată cu accesul lumii medicale și al sistemelor de sănătate la cantități enorme de date brute, abordările bazate pe învățarea profundă (*deep learning*) în diverse domenii medicale pot fi extrem de eficiente. Prin urmare, multe domenii de cercetare medicală au început să implementeze metode bazate pe învățarea automată (*machine learning*).

3.1.3 Rețele Neuronale Convoluționale

Rețelele Neuronale Convoluționale (CNN), introduse de LeCun et al. (1998), reprezintă un tip de arhitectură constând dintr-un strat de intrare (*input layer*), mai multe straturi ascunse de convoluție și de agregare și un strat de ieșire (*output layer*). Ceea ce le oferă un mare avantaj, comparativ cu rețelele neuronale profunde, sunt tipurile de straturi ascunse, care funcționează într-un mod similar cu cortexul vizual și sunt utilizate în principal pentru sarcini de recunoaștere și clasificare a imaginilor. Toate beneficiile rețelelor neuronale convoluționale pot fi atribuite complexității ridicate obținute prin reducerea dimensionalității, straturile parțial conectate și selecția automată a trăsăturilor.

3.1.4 Straturi Convoluționale

Straturile convoluționale funcționează prin aplicarea unor nuclee (*kernels*) de diferite dimensiuni, cunoscute în mod obișnuit sub numele de filtre, asupra imaginii; fiecare rezultă într-o hartă specifică de trăsături, acestea fiind suprapuse pentru o reprezentare mai bună. Neuronii din straturile anterioare sunt conectați doar la neuronii din câmpul receptiv al straturilor următoare (dimensiunea nucleului), ceea ce reduce complexitatea. Mai mult, dimensiunea matricei de pixeli de intrare poate fi redusă prin creșterea pasului, care reprezintă distanța cu care nucleul glisează între pixeli, întregul mecanism de filtrare fiind exemplificat în Figure 3.

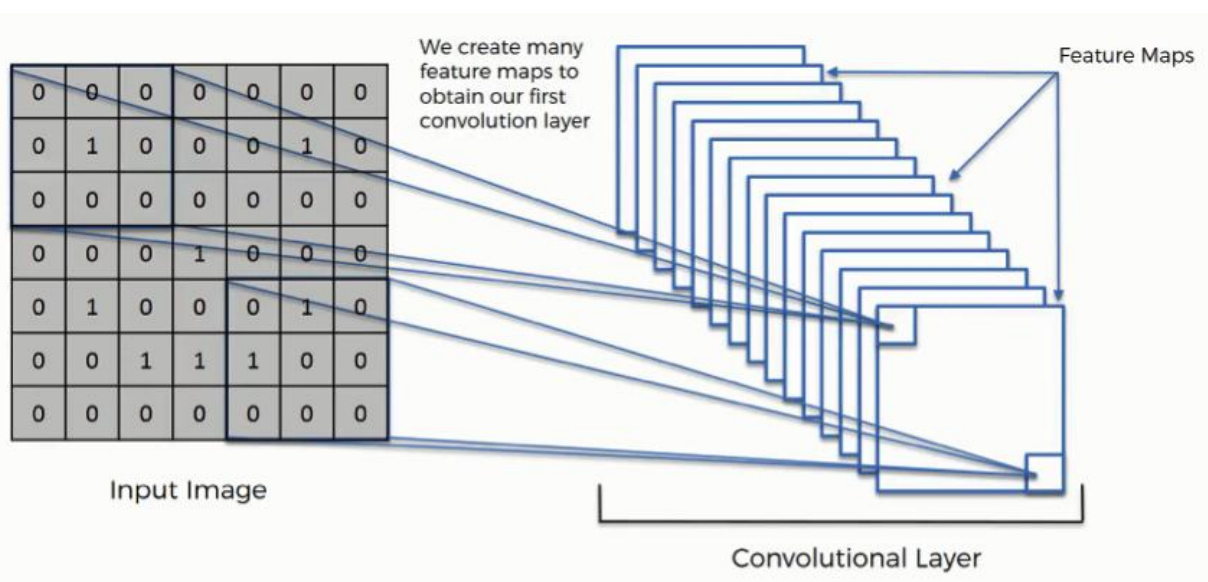


Figure 3 Exemplu de strat convolutional [3]

3.1.5 Pooling Layers

Straturile de agregare (*pooling layers*) sunt similare cu cele convoluționale, însă, în loc să realizeze produsul scalar între valorile pixelilor și matricele de ponderi ale nucleului, acestea efectuează funcții mai simple, precum media (*average*) sau maximul (*max*). Cel mai des utilizat este stratul de tip *Max Pooling*, care are ca scop reducerea dimensiunii imaginii de intrare prin returnarea exclusivă a valorii maxime a pixelului din câmpul său receptiv, procesul de reducere dimensională fiind ilustrat în Figure 4.

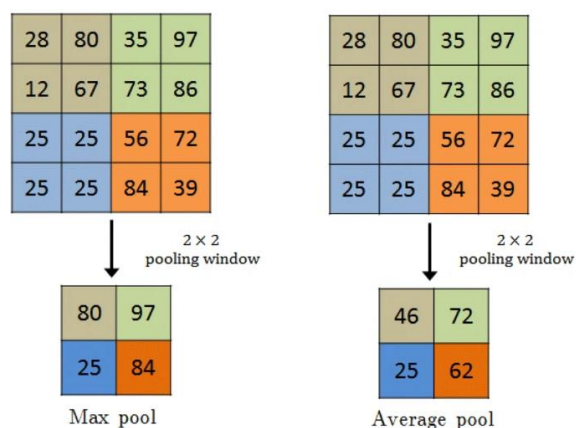


Figure 4 Exemplu de pooling layer [4]

3.1.6 Dense Layers

O strategie foarte comună în proiectarea rețelelor neuronale convoluționale este utilizarea straturilor dense, complet conectate (*fully connected layers*), ca ultimele straturi ale arhitecturii (consultați Figura 2.3). În ciuda faptului că această abordare crește complexitatea modelului, există un compromis acceptabil între pierderi și beneficii. Straturile complet conectate îmbunătățesc eficiența predicțiilor prin o mai bună „analiză” a trăsăturilor de nivel înalt care sunt cel mai puternic corelate cu o anumită clasă. Ieșirea acestora este apoi transmisă unei funcții de activare SoftMax, care returnează probabilitățile ca o anumită instanță să aparțină fiecăreia dintre clase.

3.1.7 Tranziția de la arhitecturi CNN tradiționale la arhitecturi moderne

Abordările tradiționale privind rețelele neuronale convoluționale implicau utilizarea unei serii de straturi convoluționale cu diferite funcții de activare (tanh, RBF, RELU etc.), fiecare fiind urmat

de un strat de agregare (*pooling*). Totuși, progresele recente în puterea de calcul au permis rețelelor CNN să atingă o acuratețe ridicată în clasificarea și recunoașterea imaginilor, prin abordări ușor modificate, cum ar fi suprapunerea straturilor convoluționale unul peste altul. Algoritmii câștigători ai concursului ImageNet ILSVRC (Russakovsky et al., 2015) se îmbunătățesc aproape în fiecare an, introducând arhitecturi noi și mai eficiente, acestea fiind sintetizate în Figure 5. Cu toate acestea, majoritatea modelelor implică rețele foarte profunde și costisitoare din punct de vedere computațional. Unele dintre cele mai influente lucrări care au introdus algoritmi câștigători sunt:

- AlexNet – introdusă în Krizhevsky et al. (2012), algoritm câștigător în 2012.
- GoogLeNet – introdusă în Szegedy et al. (2015), algoritm câștigător în 2014.
- ResNet – introdusă în He et al. (2016), algoritm câștigător în 2015.

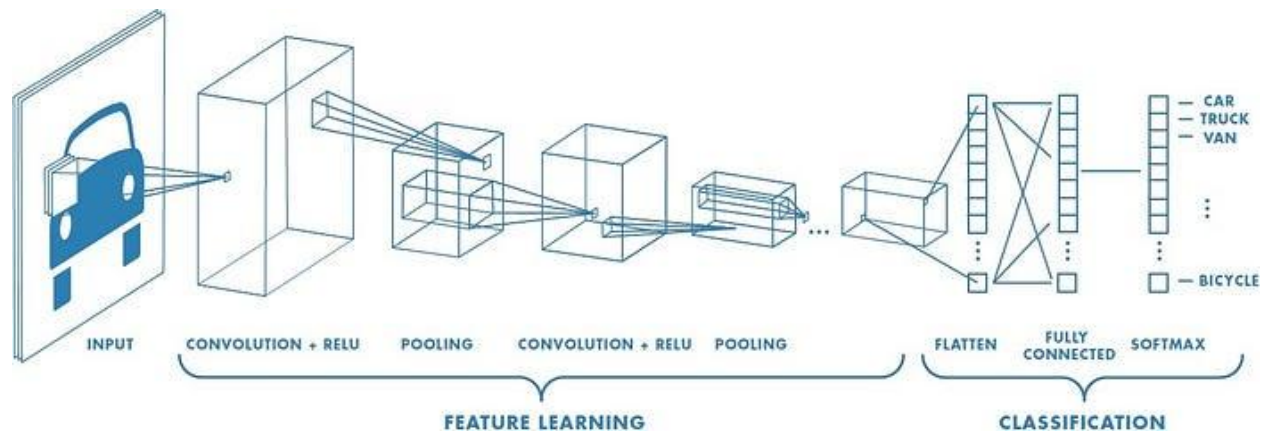


Figure 5 Exemplu de CNN [5]

3.1.8 Provocări în antrenarea rețelelor CNN

În ciuda faptului că rețelele neuronale convoluționale sunt capabile să obțină rezultate excelente în problemele de clasificare a imaginilor, procesul de antrenare a rețelei pentru a generaliza corect pe instanțe noi, neprevăzute, prezintă numeroase dificultăți.

3.1.8 Hiperparametrii și optimizatorul

[26] În cadrul problemei abordate, atunci când se antrenează rețele neuronale convoluționale, este esențial ca anumiți parametri să fie setați corespunzător, cum ar fi:

dimensiunea lotului (*batch size*), numărul de epoci, optimizatorul, împreună cu rata de învățare a acestuia și funcția de cost (*loss function*). În domeniul învățării automate, parametrii care trebuie stabiliți înainte de începerea procesului de antrenare sunt cunoscuți sub numele de hiperparametri.

Dimensiunea lotului (*batch size*) reprezintă numărul de instanțe de antrenament utilizate într-un singur pas de actualizare a ponderilor. Atunci când procesul de antrenare este realizat folosind unități de procesare grafică (plăci video - GPU), dimensiunea imaginilor din lot trebuie să se încadreze în memoria RAM disponibilă a acestora.

[28] În al doilea rând, numărul de epoci reprezintă de câte ori algoritmul parcurge întregul set de date pe durata procesului de învățare. Acesta trebuie setat la o valoare suficient de mare pentru ca algoritmul să nu sufere de sub-ajustare (*underfit*) pe setul de date de antrenament, dar și suficient de mică pentru a preveni supraantrenarea (*overfitting*). În timpul procesului de antrenare, numărul de pași dintr-o epocă este de obicei stabilit ca fiind raportul dintre numărul total de instanțe de antrenament și dimensiunea lotului, astfel încât algoritmul să poată „vedea” toate imaginile din setul de date de antrenament pe parcursul fiecărei epoci.

Funcția care evaluează performanța algoritmului se numește funcție de cost (*loss function*). În cea mai simplă formă a sa, aceasta calculează diferența dintre valoarea prezisă și valoarea reală, deoarece pierderea (*loss*) reprezintă magnitudinea erorii.

În cele din urmă, optimizatorul într-un model de învățare automată este funcția care corelează parametrii modelului cu funcția de cost. Cel mai popular optimizator se numește descendența gradientului (*gradient descent*). Ideea principală din spatele acestor instrumente este că, în funcție de modul în care evoluează valoarea funcției de cost — fie în scădere, fie în creștere optimizatorul știe dacă se îndreaptă în direcția corectă (către o soluție mai bună) sau dacă, dimpotrivă, diverge de la calea optimă.

Un hiperparametru foarte important atunci când se alege un optimizator este rata de învățare (*learning rate*). Dacă rata de învățare a optimizatorului este setată la o valoare prea mare, algoritmul poate progresa bine la început, dar nu va converge niciodată către cea mai bună soluție. La polul opus, dacă rata de învățare este setată prea mic, algoritmului îi poate lua foarte mult timp să convergă către o soluție bună. Prin urmare, rata de învățare este un parametru foarte sensibil care trebuie setat cu atenție, impactul diferitelor sale valori asupra convergenței fiind sintetizat în Figure 6. Ajustarea corectă se realizează în principal prin testarea diferitelor valori și utilizarea celei care oferă cele mai bune performanțe.

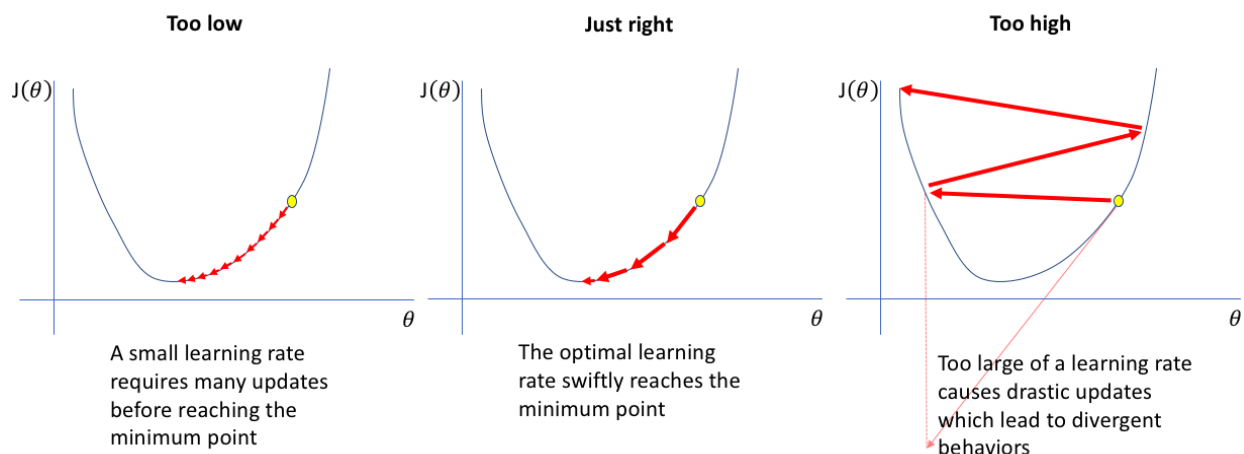


Figure 6 Exemplu de learning rate [6]

3.1.9 Overfitting, Underfitting și Dropout

[18] Problema care apare cel mai frecvent în timpul antrenării rețelelor neuronale profunde se numește supraantrenare (*overfitting*). Atunci când un model de învățare automată supraantrenează setul de date de antrenament, acest lucru înseamnă că algoritmul mai degrabă memorează instanțele de antrenament decât învață din ele. Dacă acest lucru se întâmplă, algoritmul nu este capabil să generalizeze suficient de bine, rezultând un model ineficient (Géron, 2017). Pentru rezolvarea acestei probleme, ar trebui utilizate tehnici de regularizare aplicate algoritmului.

Problema opusă este cunoscută sub numele de sub-ajustare (*underfitting*) și apare atunci când implementarea algoritmului nu este adecvată pentru problemă, nu este suficient de complexă pentru a o rezolva sau atunci când nu sunt disponibile suficiente date de antrenament (consultați Figure 7).

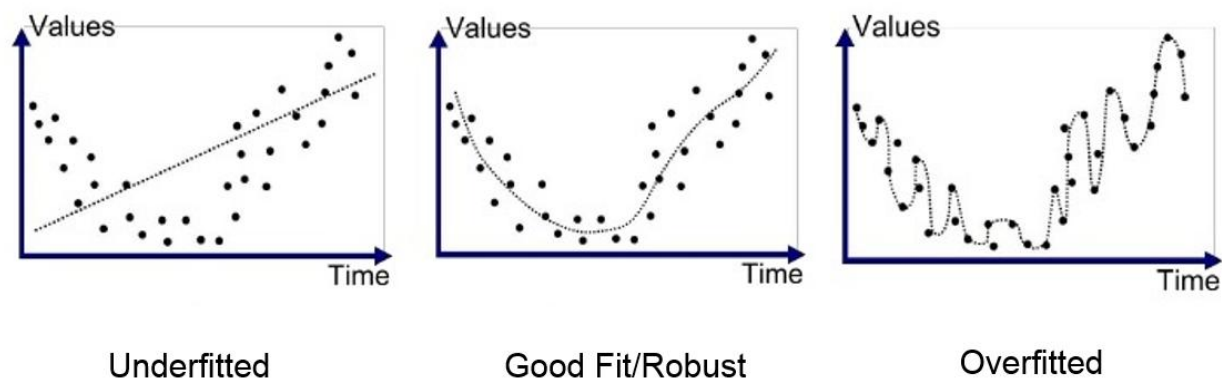


Figure 7 Exemplu de overfitting si underfitting [7]

3.1.10 Augmentarea Datelor

[21] O metodă alternativă care poate fi aplicată pentru a îmbunătăți performanța algoritmilor se numește augmentarea datelor. Atunci când se aplică această procedură, instanțele de antrenament sunt modificate în moduri distincte, astfel încât algoritmul să fie antrenat pe mai multe versiuni diferite ale aceleiași instanțe. Metodele aplicate la modificarea imaginilor includ: rotirea, schimbarea luminozității, mărirea (*zooming*), oglindirea (*flipping*) sau deplasarea acestora, astfel de proces de transformare fiind exemplificat în Figure 8. Această metodă este utilă și atunci când se lucrează cu seturi de date de dimensiuni mici, deoarece poate fi utilizată pentru a crește numărul de instanțe prin crearea unor versiuni modificate ale acestora. Totuși, procedurile de augmentare trebuie alese cu atenție pentru a nu pierde prea multe detalii (Géron, 2017).

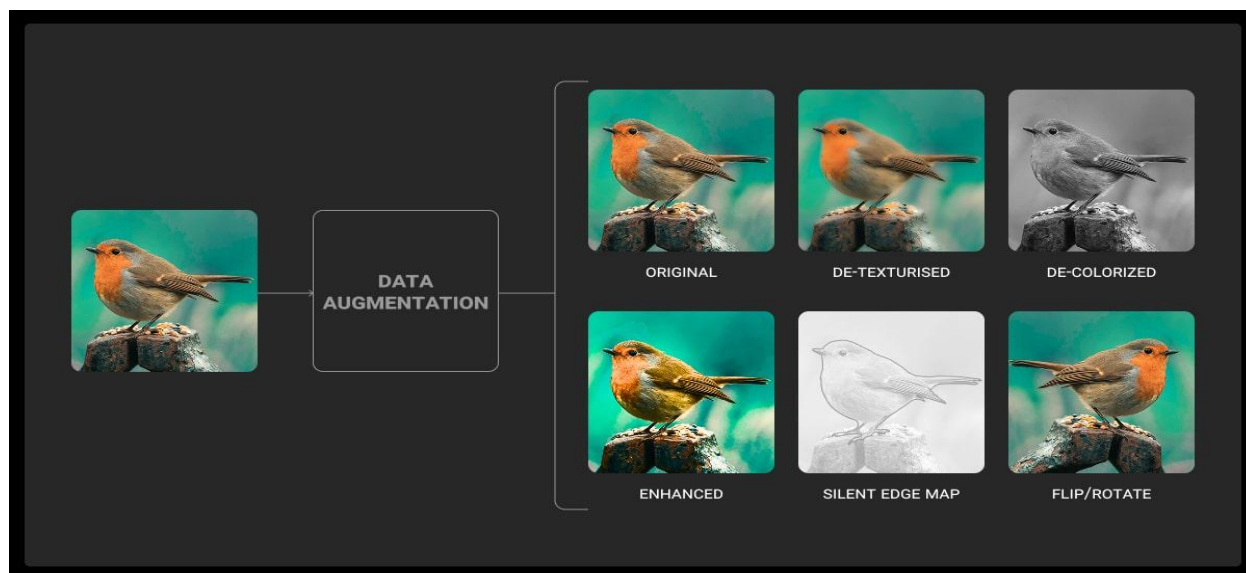


Figure 8 Exemplu de augmentare de date [8]

3.1.11 Problema “Vanishing gradient”

Fenomenul de „Vanishing Gradient” (Dispoziția sau Atenuarea Gradientului) în zonele extreme ale funcției Swish este una dintre cele mai fine probleme de natură matematică pe care le poți discuta în cadrul unei licențe de inginerie.

În rețelele neuronale, ajustarea ponderilor (adică modul în care rețeaua învață din greșeli) se face pe baza derivatei funcției de activare. Derivata ne spune, practic, panta curbei: cât de mult se modifică ieșirea unui neuron dacă modificăm foarte puțin intrarea.

Formula funcției Swish este:

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (1.0)$$

Impactul direct al acestui fenomen asupra comportamentului modelului pe parcursul experimentelor este ilustrat în Figure 9 manifestându-se prin următoarele aspecte tehnice :

1. Plafonarea acurateței: se va observa pe graficele de antrenare că modelul pornește bine, dar după câteva epoci, acuratețea pe setul de validare se blochează complet (de exemplu, la 48% în cazul meu) și refuză să mai crească, chiar dacă îl mai lași să ruleze încă 20 de epoci. Mulți dintre neuronii din straturile profunde au „alunecat” în zonele de saturație și au încetat să mai învețe, lăsând restul rețelei să ducă greul.
2. Sensibilitate la rata de învățare: dacă rata mea de învățare ar fi fost prea mare (de exemplu, 10^{-2}) ponderile ar fi fost modificate violent de la primii pași direct în zonele de saturație extremă, blocând rețeaua instantaneu. Acesta este motivul tehnic ascuns pentru care am fost forțat să folosesc o rată de învățare extrem de mică (5×10^{-5}). O rată mică menține activările în zona centrală a funcției Swish, unde derivata are valori mari și rețeaua poate face pași siguri și stabili către optimul global.

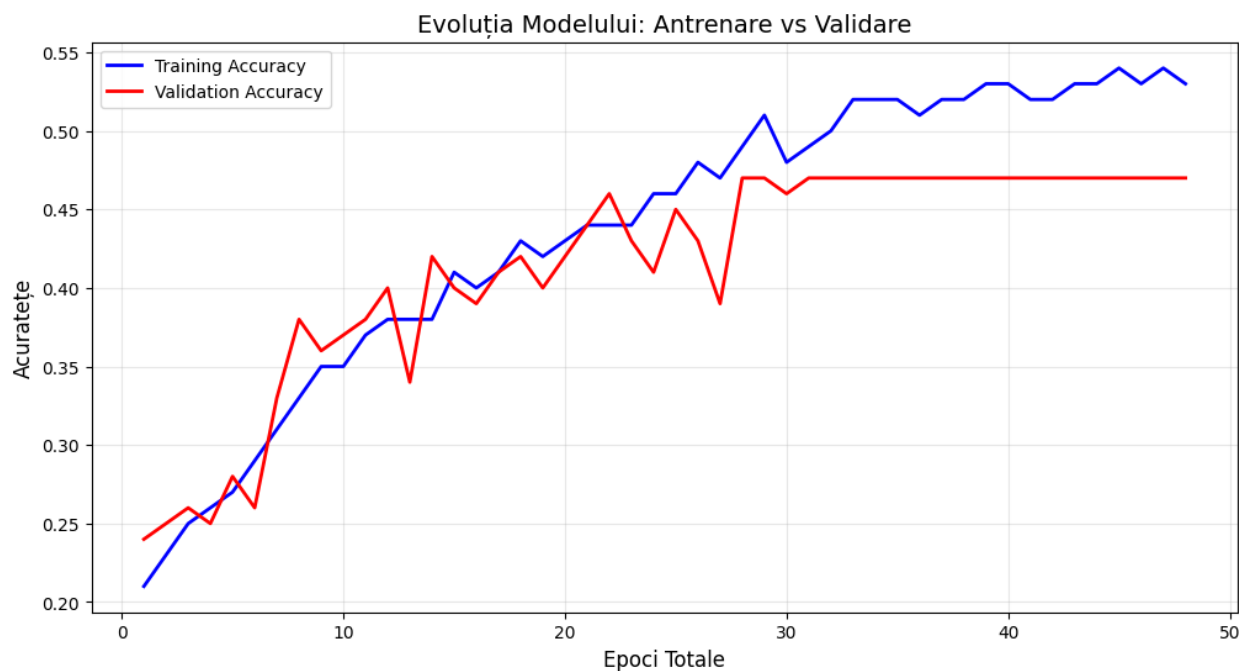


Figure 9 Apariția problemei de vanishing gradient [9]

3.1.12 Transfer Learning

[29] În cele din urmă, o altă metodă utilizată foarte frecvent în domeniul învățării profunde și al rețelelor neuronale convoluționale este cunoscută sub numele de învățare prin transfer (*transfer learning*). Ideea principală din spatele acestei tehnici este că, indiferent de problema pe care cineva încearcă să o rezolve, o altă persoană a încercat deja anterior să abordeze o problemă similară. Prin urmare, primul lucru de făcut atunci când se începe lucrul la o problemă de învățare automată este verificarea lucrărilor și a studiilor anterioare realizate pe teme conexe. După aceea, se verifică tipul de arhitectură utilizat și, în loc de a proiecta întregul model de învățare automată de la zero, se abordează problema folosind același model. Dacă problema este foarte asemănătoare, procesul de antrenare poate fi lansat chiar și cu ponderi pre-antrenate (*pretrained weights*) din modelul implementat pentru problema înrudită.

De regulă, atunci când se dezvoltă un model de învățare profundă bazat pe rețele neuronale convoluționale, se utilizează arhitecturi care s-au dovedit a fi foarte eficiente prin performanțele obținute la competiția ImageNet ILSVRC menționată anterior (consultați Secțiunea 2.3.4). Mai mult, prin utilizarea modelelor pre-antrenate și adăugarea doar a câtorva straturi spre finalul arhitecturii pentru a o adapta la problema curentă, procesul de antrenare va fi mult mai eficient din punctul de vedere al timpului și poate necesita, de asemenea, mult mai puține date (Géron, 2017).

În consecință, învățarea prin transfer este, probabil, cea mai puternică și populară tehnică utilizată în domeniul învățării profunde.

3.1.13 Lucrari inrudite

În ceea ce privește lucrările înrudite, în trecut au fost realizate numeroase studii asociate cu problema clasificării stadiilor retinopatiei diabetice folosind metode de învățare automată. O sinteză a activității anterioare desfășurate în acest domeniu, precum și a metodelor, rezultatelor și seturilor de date utilizate în fiecare dintre acestea, este prezentată de Islam et al. (2020). Unele dintre cele mai relevante lucrări pentru prezentul studiu vor fi discutate în continuare mai detaliat.

În lucrarea lor, Chang et al. (2018) au utilizat setul de date disponibil public din competiția Kaggle din 2015 pentru a dezvolta un model de învățare automată care clasifică fazele retinopatiei diabetice. Setul de date furnizează 35.126 de imagini ale retinei umane, diagnosticate de clinicieni certificați în 5 stadii de severitate diferite (de la 0 la 4), clasa 0 fiind cea mai puțin severă, iar imaginile aparținând clasei 4 având cea mai mare gravitate. Aceștia au ales să reformuleze problema, transformând-o într-una de clasificare binară: imaginile din clasa 0 au fost modificate pentru a aparține clasei „Sănătos”, iar imaginile din clasele 2, 3 și 4 au fost categorisite ca „Bolnav”. Instanțele aparținând clasei 1 au fost eliminate complet. În ceea ce privește rezoluția

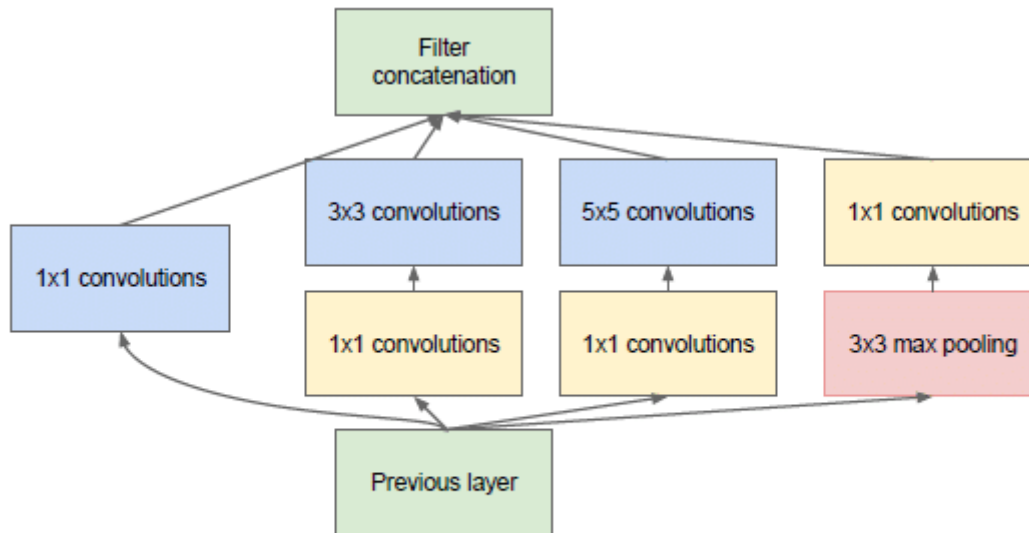
imaginilor în timpul procesului de antrenare, aceștia au ales să le redimensioneze la 256x256 pixeli. O altă alegere importantă de proiectare discutată în lucrare a fost preprocesarea imaginilor folosind o tehnică implementată de câștigătorul competiției Kaggle din 2015. Pentru a preveni supraantrenarea, au fost implementate tehnici de augmentare a datelor utilizând metoda integrată disponibilă în biblioteca Keras. Imaginile au fost rotite aleatoriu și oglindite pe orizontală. În ceea ce privește algoritmul, aceștia au folosit învățarea prin transfer, dezvoltând modelul prin reutilizarea arhitecturii cunoscute sub numele de ResNet34. Pentru dezvoltarea clasificatorului, a fost utilizată o placă video NVIDIA P100. Referitor la evaluare, acuratețea a fost utilizată ca metrică de performanță, obținându-se un procent de 78,7% pe subsetul de testare.

Un alt studiu care s-a concentrat pe dezvoltarea unui algoritm pentru clasificarea stadiilor bolii a fost realizat de Pratt et al. (2016). Pe parcursul dezvoltării modelului a fost utilizat același set de date ca în lucrarea discutată anterior. În loc să reducă problema la una de clasificare binară, scopul acestui studiu a fost crearea unui clasificator multiclăsă, care să ia în considerare toate cele cinci faze de gravitate. Referitor la resursele hardware și software, implementarea a fost realizată pe o placă grafică NVIDIA K40c, profitând de biblioteca Keras pentru Python. Autorii au ales să preproceseze imaginile prin normalizarea culorilor pentru a contracara diferențele mari de iluminare dintre instanțe. Mai mult, imaginile au fost redimensionate la o rezoluție de 512x512 pixeli din cauza limitărilor plăcii grafice. Arhitectura exactă a modelului nu este menționată, însă abordarea s-a bazat pe rețele neuronale convoluționale. Similar studiului precedent, datele au fost augmentate prin rotirea aleatorie a imaginilor și oglindirea acestora pe orizontală sau verticală pentru a depăși supraantrenarea. Rezultatele raportate sunt: specificitate de 95%, sensibilitate de 30% și o acuratețe de 75%. Analiza matricei de confuzie a relevat faptul că niciuna dintre imaginile aparținând clasei de severitate 1 nu a fost prezisă ca atare. Concluzia lor subliniază potențialul metodelor de învățare profundă în diagnosticare, menționând totodată necesitatea unui volum mai mare de date de calitate superioară.

Un studiu mai recent care a investigat clasificarea stadiilor de gravitate a fost condus de Sayres et al. (2019). O îmbunătățire notabilă observată în lucrarea lor a fost dimensiunea setului de date utilizat, acesta constând în 140.000 de imagini furnizate de clinicile EyePacs și alte 3 spitale din India, evaluate de specialiști în retină. O altă îmbunătățire foarte importantă a fost utilizarea imaginilor la rezoluții de 779x779 pixeli, oferind o creștere semnificativă a calității.

Informațiile despre hardware nu sunt furnizate, însă, având în vedere rezoluția ridicată și numărul mare de instanțe din setul de date, se poate presupune că au fost disponibile specificații foarte puternice. Mai mult, modelul de învățare profundă s-a bazat pe o arhitectură foarte performantă și populară dezvoltată de Google, cunoscută sub numele de Inception v4, blocul structural fundamental al acestui tip de rețea fiind detaliat în Figure 10. Matricea de confuzie raportată în secțiunea de rezultate a lucrării lor indică o acuratețe de 88,4% în clasificarea celor cinci stadii de gravitate ale retinopatiei diabetice. Totuși, o observație importantă a fost numărul foarte mic de instanțe păstrate pentru validare și distribuția neuniformă a acestora. Doar 1800 de instanțe din cele 140.000 sunt raportate în matricea de confuzie, cu doar 30 de imagini pentru

fiecare dintre clasele de gravitate 3 și 4. De asemenea, la fel ca în lucrarea discutată anterior, cele mai mari probleme au apărut la prezicerea instanțelor cu eticheta reală 1, deoarece multe dintre acestea sunt interpretate ca aparținând fie clasei 0, fie clasei 2.



(b) Inception module with dimensionality reduction

Figure 10 Exemplu de inception modul [10]

3.1.14 Resursele pentru elaborarea proiectului

Următoarea decizie importantă s-a concentrat pe rezoluția imaginilor. Inițial, imaginile aveau rezoluții de aproximativ 1024x1024 pixeli, ceea ce s-a dovedit a fi mult peste limitările tehnice ale proiectului. Deși antrenamentul a fost efectuat pe platforma Kaggle, utilizând unități de procesare grafică puternice, precum NVIDIA Tesla P100 (cu 16 GB memorie RAM HBM2), timpii de antrenare ar fi fost sever afectați de rezoluții atât de mari. Prin urmare, imaginile au fost redimensionate la o rezoluție de 320x320 pixeli pentru a optimiza procesul de învățare fără a suprasolicita resursele hardware disponibile.

Alte decizii importante de proiectare au vizat parametrii necesari pentru ca algoritmul să poată „învăța” din datele de antrenament și să generalizeze corect pe instanțe noi, neprevăzute. Fluxul complet de realizare a aplicației, alături de distribuția procentuală a setului de date, este schițat în Figure 11 respectiv Figure 12. Printre aceștia, cei care au cântărit cel mai mult în performanța algoritmului sunt: funcția de cost, dimensiunea lotului (*batch size*), optimizatorul și rata de învățare. Modelele au fost compilate folosind funcția de cost *Categorical Cross Entropy*,

care oferă rezultate optime în problemele de clasificare multclasă. Optimizatorul *Adaptive Moment Estimation*, cunoscut sub numele de Adam, a fost principalul instrument utilizat pe parcursul implementării modelelor.

În ceea ce privește dimensiunea lotului să se încadreze în memoria RAM disponibilă pe placa grafică. Deși au fost încercate mai multe configurații, valoarea predominantă utilizată a fost 16. Rata de învățare este descrisă pe larg în secțiunea de Implementare, alături de explicații mai detaliate ale parametrilor deja menționați.

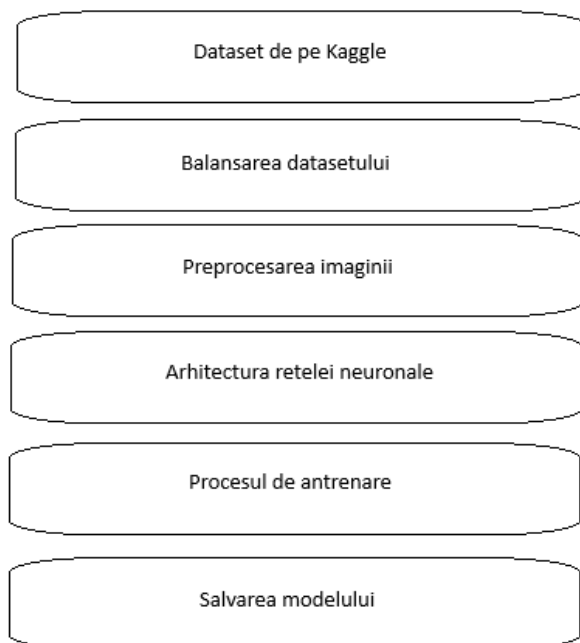


Figure 11 Un sketch ce arată pașii pentru crearea aplicației [11]

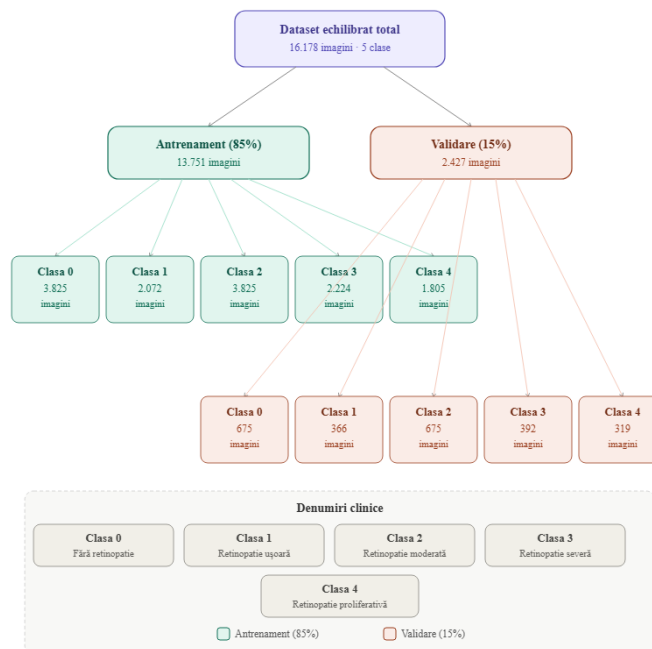


Figure 12 Distribuția setului de date [12]

3.2 Implementare

3.2.1 Hardware și software

În primul rând, vor fi menționate resursele hardware și software utilizate pe parcursul dezvoltării acestui proiect.

În ceea ce privește componenta hardware, procesul de antrenare a modelului a fost realizat utilizând platforma Kaggle, având un GPU numit NVIDIA P100. Această putere de calcul a fost esențială pentru gestionarea eficientă a volumului mare de date și pentru reducerea timpului de antrenare a rețelei EfficientNetB0. Pe de altă parte, etapa de implementare fizică și inferență în timp real a fost realizată pe un sistem embedded mai exact plăcuța NVIDIA Jetson Nano Developer Kit 2GB (echipată cu un procesor ARM Cortex-A57 și un GPU bazat pe arhitectura NVIDIA Maxwell cu 128 de nuclee CUDA).

Limbajul de programare principal utilizat pentru execuția și dezvoltarea modelului de învățare automată este Python. Motivele din spatele acestei alegeri sunt strâns legate de ecosistemul bogat de librării specializate, care simplifică și eficientizează implementarea algoritmilor complecși. Pentru construirea, antrenarea și salvarea rețelei neuronale au fost utilizate framework-urile TensorFlow și Keras. De asemenea, codul sursă pentru antrenare și

analiză statistică a datelor a fost dezvoltat direct pe platforma Kaggle, deoarece avem acces la GPU destinat proiectelor de machine learning/ AI, ceea ce ne va ajuta sa avem un timp de antrenare redus.

În ceea ce privește aplicația de rulare live, implementarea s-a mutat pe dispozitivul embedded Jetson Nano, ce face procesarea imaginilor direct local. Integrarea aplicației (inclusiv versiunile pentru TensorFlow și Keras) a fost asigurată prin virtualizare utilizând containere Docker. Pentru preluarea fluxului video de la camera web și afișarea interfeței grafice direct pe ecran, s-a utilizat librăria OpenCV.

3.2.2 Nvidia Jetson Nano Developer Kit

[20] Kitul de dezvoltare NVIDIA® Jetson Nano™ 2GB este ideal pentru predarea, învățarea și dezvoltarea aplicațiilor de inteligență artificială (AI) și robotică. Beneficiind de o comunitate activă de dezvoltatori și de proiecte open-source gata de implementat, veți găsi toate resursele necesare pentru a începe. Acesta oferă performanțe AI remarcabile la un preț redus, făcând lumea inteligenței artificiale și a roboticii accesibilă tuturor, prin utilizarea exact a acelorași instrumente și medii software NVIDIA folosite pentru crearea produselor AI inovatoare în toate domeniile industriale.

Placa Jetson Nano 2GB este un modul de calcul compact, echipat cu un GPU Maxwell de 128 de nuclee, proiectat pentru a facilita procesarea de mare viteză a modelelor de inteligență artificială. Acesta este deosebit de bine adaptat pentru sarcini solicitante din punct de vedere computațional, cum ar fi clasificarea imaginilor, detecția obiectelor și segmentarea. Suportând atât carduri microSD, cât și unități SSD ca medii de stocare, Jetson Nano funcționează într-un interval redus de consum de energie, cuprins între 5 W și 10 W. Aceste caracteristici îl fac extrem de avantajos pentru sistemele care necesită o funcționare prelungită și continuă, fără a depinde de resurse energetice substanțiale. Placa a fost construită de către compania NVIDIA, așa cum este ilustrat în Figure 13.



Figure 13 Placa Nvidia Jetson Nano Developer Kit [13]

3.2.3 Analiza interfețelor de conectivitate și periferice pentru JetsonNano

Vederea frontală a kitului de dezvoltare NVIDIA Jetson Nano 2GB grupează principalele interfețe de intrare/ieșire (I/O) destinate alimentării, transferului de date de mare viteză și redării video. Configurația acestui panou de conectori este esențială pentru stabilirea topologiei sistemului de diagnosticare live.

1. Conectorul de alimentare USB Type-C: spre deosebire de versiunea standard de Jetson Nano (4GB) care utilizează o mufă de tip Barrel Jack, versiunea de 2GB integrează un port USB Type-C dedicat exclusiv alimentării cu energie electrică a sistemului iar acesta suportă o tensiunea de 5V la care este stabil cu o intensitate a curentului de 3 A având 15W în total. Utilizarea unui alimentator Type-C de calitate a fost important în acest proiect pentru a putea selecta modul de performanță ridicată MAXN (10W). În momentul de full load când programul rulează, cele 128 de nuclee CUDA și cele 4 nuclee CPU rulează la frecvență maximă, se poate ca placa să-și dea singura restart, de aceea având un acumulator bun ce poate trage mai mult sau 3V este esențial.
2. Portul Video HDMI Type-A: permite transmiterea semnalului video către monitoare externe, suportând rezoluții de până la 4K cu un refresh rate de 60Hz. Acest port a fost utilizat de către mine pentru conectarea unui monitor extern pentru a putea vedea interfața grafică OpenCV.

3. Portul USB 3.0 Type-A (culoare albastră): oferă o lăţime de bandă considerabilă şi un management superior al energiei livrate perifericelor mari.
4. Porturile USB 2.0 Type-A (x2): sunt destinate perifericelor cu cerinţe reduse de lăţime de bandă.
5. Interfaţa de Reţea Gigabit Ethernet (RJ-45): suportă viteze de transmisie de 10/100/1000 Mbps (Gigabit Ethernet). Deşi rularea modelului EfficientNetB0 şi diagnosticul retinopatiei se execută 100% local în interiorul containerului Docker (fără a trimite imaginile pe internet), conexiunea Ethernet a fost esenţială în faza de configurare pentru descărcarea pachetelor pentru a putea rula programul (cv2, pip) prin apt-get.
6. Portul MicroUSB: funcţionează în modul Device/Device Mode (L4T). Atunci când placa Jetson Nano este conectată prin acest port la un PC gazdă prin cablu, ea emulează o placă de reţea virtuală (alocând automat IP-ul 192.168.55.1).
7. Slotul pentru Cardul MicroSD: spre deosebire de computerele clasice care utilizează unităţi de stocare de tip HDD sau SSD conectate prin magistrale SATA/NVMe, Jetson Nano 2GB utilizează un slot de card MicroSD ca mediu principal de stocare pentru sistemul de operare.

3.2.3 Setul de date

Setul de date utilizat pe parcursul etapelor de antrenare, validare şi testare ale modelului a fost construit prin unirea imaginilor provenite din două competiţii organizate anterior pe platforma Kaggle.

Prima competiţie a fost „2015 Blindness Detection” şi a fost compusă din 35 de mii de imagini destinate etapei de antrenare şi 50 de mii de imagini pentru testare, în timp ce a doua competiţie a oferit doar 3500 de imagini pentru antrenare şi 3500 pentru testare.

Imaginile au fost furnizate în format .jpeg, având rezoluţii de aproximativ 1024x1024 pixeli, împreună cu etichetele reale (ground truth) oferite în format .csv. În ceea ce priveşte calitatea, aceasta a variat semnificativ de la o imagine la alta, deoarece dispozitivele utilizate pentru achiziţia lor nu au fost la fel de performante, existând de asemenea diferenţe de iluminare şi de poziţionare.

Procesul de analiză exploratorie a datelor a reprezentat primul pas realizat în pipeline-ul de implementare. O primă observaţie a fost aceea că etichetele reale lipseau pentru lotul de imagini de testare din competiţia din 2019, fapt care le-a exclus din scopul unui algoritm de învăţare supervizată, cum sunt reţelele neuronale convoluţionale. În continuare, a fost efectuată investigarea celor 35.000 de imagini din setul de antrenare al competiţiei din 2015. După crearea

unui set de date care asocia numele fiecărei fotografii cu stadiul său de severitate, s-a observat rapid o diferență majoră în ceea ce privește numărul de instanțe dintre clase, peste 70% din imagini aparținând clasei 0. Restul de 30% nu erau, de asemenea, distribuite în mod uniform, existând doar aproximativ 800 de imagini aparținând fiecăreia dintre clasele 3 și 4 – adică în jur de 3 procente din totalul setului de date. Acest lucru a generat o problemă extrem de dificilă în implementarea algoritmului, seturile de date dezechilibrate reprezentând una dintre cele mai mari provocări în dezvoltarea unui model de învățare automată.

După o analiză inițială a distribuției, au fost efectuate niste modificări asupra setului de date cu scopul de a avea un echilibru în setul nostru de date. Pentru a obține o reprezentare optimă, am aplicat strategia de sub-eșantionare (*undersampling*) și supra-eșantionare (*oversampling*). În ceea ce privește clasele 0 și 2, s-a optat pentru limitarea acestora având un de maximum 4500 de poze pentru fiecare dintre aceste clase, prevenind astfel supraspecializarea rețelei pe cazurile comune. Pentru clasa 1, fiind disponibil un număr total de 2438 de imagini au fost utilizate toate pozele existente. Iar pentru clasele severe și critice, respectiv clasele 3 (Retinopatie Severă) și 4 (Retinopatie Proliferativă), s-a aplicat o tehnică de multiplicare directă, seturile lor de date fiind înmultite de 3 ori. Acest lucru a ajutat la o distribuție mult mai echilibrată și omogenă între cele 5 clase pentru setul de date final, ceea ce a îmbunătățit performanțele generale ale procesului de clasificare.

3.2.4 Arhitectura modelului bazat pe Transfer Learning și prevenirea supraantrenării (Overfitting)

[24] Pentru a obține o performanță ridicată în clasificarea stadiilor retinopatiei diabetice, fără a fi necesară o putere de calcul colosală dedicată antrenării unei rețele de la zero, s-a optat pentru tehnica de Transfer Learning. În locul unui design experimental simplu, a fost utilizată arhitectura preantrenată aceea fiind EfficientNetB0, recunoscută în literatura de specialitate pentru eficiența sa computațională brută și pentru scalarea inteligentă a adâncimii, lățimii și rezoluției rețelei prin coeficienți compuși. Procesul de optimizare a vizat adaptarea acestui model robust (antrenat inițial pe setul de date ImageNet) la specificul imaginilor medicale ale fundului de ochi.

Folosind rețeaua EfficientNetB0, s-a aplicat o strategie de reglaj fin parțial. Primele 40 de straturi ale rețelei de bază au fost „înghețate” (`layer.trainable = False`) pentru a conserva extractorul general de caracteristici (linii, muchii, texturi), în timp ce restul straturilor superioare au fost lăsate deschise reantrenării pentru a învăța tiparele complexe ale patologiei retiniene (microanevrisme, exsudate). Toate instanțele de antrenare introduse în rețea au fost configurate la o rezoluție fixă de 320x320 pixeli, formatul ideal pentru această variantă de arhitectură.

Pentru a mapa caracteristicile extrase către cele 5 clase de severitate medicală și pentru a combate fenomenul de *overfitting* (supraspecializare pe datele de antrenare), deasupra modelului de bază au fost atașate următoarele straturi customizate:

1. GlobalAveragePooling2D: ia toate hărțile de caracteristici generate de rețea și le "comprimă" într-un singur număr per hartă, calculând media valorilor. Practic, din sute de imagini mici cu informații, iese un singur vector compact — mai ușor de procesat și mai puțin predispus la memorarea datelor de antrenament (overfitting).
2. BatchNormalization: ajuta la stabilizarea procesului de antrenare
3. Dense(512, activation="swish"): un strat complet conectat cu 512 neuroni care utilizează funcția de activare modernă *Swish*, superioară funcției standard ReLU în rețelele adânci.
4. Dropout(0.6): un strat care dezactivează aleatoriu 60% din conexiunile neuronilor la fiecare pas al antrenării, forțând rețeaua să nu învețe pe de rost pozele (overfitting)
5. Dense(5, activation="softmax"): acesta este stratul final de ieșire compus din 5 neuroni, pentru cele 5 stadii ale bolii. Acesta utilizează funcția de activare SoftMax pentru a transforma valorile brute calculate într-o distribuție probabilistică (procente de certitudine).

În final, modelul a fost compilat folosind funcția de Sparse Categorical Crossentropy. Am folosit aceasta funcție deoarece labelurile celor 5 clase sunt numere întregi (0,1,2,3,4) și nu sub forma de vectori de tip one hot encoding, optimizând astfel consumul de memorie RAM în timpul procesului de calcul pe platforma Kaggle. Formularea matematică a funcției de activare *Softmax* și a celei de *Cross-Entropy*, care guvernează acest ultim stadiu al rețelei, este prezentată detaliat în Figure 14. Optimizatorul folosit este Adam, cu o rată de învățare foarte fină, ea fiind setată la 5×10^{-5} .

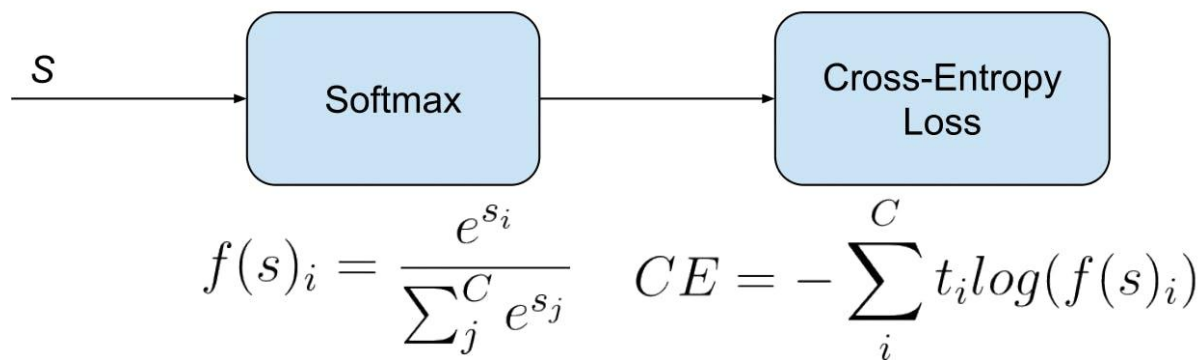


Figure 14 Sparse Categorical Crossentropy [14]

Fenomenul de supraantrenare (*overfitting*) apare atunci când modelul statistic de învățare automată învață setul de date de antrenare atât de bine, încât performanța sa pe seturi de date noi (nemaîîntâlnite) este una slabă. Cu alte cuvinte, acest lucru înseamnă că valorile prezise corespund mult prea fidel valorilor reale observate în setul de date de antrenare, generând ceea ce este cunoscut sub numele de *overfitting*. Supraantrenarea se produce atunci când un model statistic de învățare automată învață atât componentele sistematice, cât și zgomotul (fluctuațiile aleatorii) din datele de antrenare, într-o măsură care afectează negativ performanța modelului pe date noi. Acest

lucru înseamnă că modelul statistic se adaptează foarte bine atât la zgomot, cât și la semnalul util prezent în datele de antrenare, discrepanța de performanță dintre setul de testare și cel de antrenare fiind ilustrată grafic în Figure 15.

Problema constă în faptul că aceste concepte învățate nu se aplică datelor independente (noi) și afectează în mod negativ capacitatea de generalizare a modelului. Supraantrenarea este mult mai probabilă atunci când se optimizează o funcție de cost în cadrul unui model statistic complex (cu un grad mai mare de flexibilitate). Din acest motiv, multe modele statistice neparametrice de învățare automată includ, de asemenea, constrângeri în cadrul funcției de cost pentru a îmbunătăți procesul de învățare. De exemplu, rețelele neuronale artificiale (ANN), menționate ulterior, reprezintă un model statistic neparametric extrem de flexibil, care este predispus la supraantrenarea datelor de antrenare. Această problemă poate fi abordată prin aplicarea tehnicii de *dropout* (anularea sau setarea pe zero) a ponderilor unui anumit procent de unități ascunse (neuroni), cu scopul de a evita fenomenul de *overfitting*.

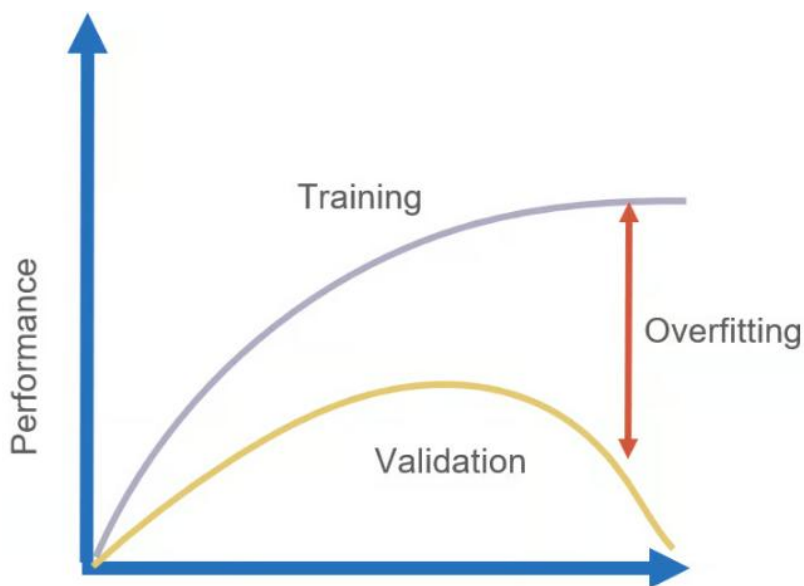


Figure 15 Overfitting intr-un model: test vs train plot [15]

3.2.5 Regularizarea Rețelei Neuronale Convoluționale

[30] Primul pas în procesul de îmbunătățire a modelului cu scopul de a preveni fenomenul de supraantrenare (*overfitting*) a fost adăugarea metodelor de augmentare a imaginilor. Metoda `ImageDataGenerator` disponibilă în librăria `Keras` oferă o modalitate de augmentare a datelor în timp real. Au fost cercetate diverse moduri de alterare a imaginilor. Tehnicile care s-au dovedit a fi utile – prin asigurarea unor variații ale aceleiași instanțe de antrenare pe parcursul diferitelor epoci, fără a degrada în mod dăunător imaginea – au fost: ajustarea intervalului de luminozitate

(*brightness range*), intervalul de zoom (*zoom range*), forfecarea (*shear range*), intervalul de rotație (*rotation range*), precum și inversarea orizontală (*horizontal flip*) și verticală (*vertical flip*). Aceste modificări au condus la un set de date mărit artificial, fapt care este extrem de important pentru o mai bună învățare și pentru capacitatea de generalizare a modelului de învățare automată.

[25] Următoarea îmbunătățire a implicat adăugarea straturilor de tip *dropout*. Așa cum s-a menționat anterior, tehnica dropout funcționează prin ignorarea completă a unei proporții de neuroni din straturile în care este utilizată pe parcursul procesului de antrenare, o astfel de decuplare structurală fiind exemplificată în Figure 16. Proporția de neuroni este stabilită prin alegerea unei valori pentru hiperparametrul „p” al metodei dropout. O valoare inițială optimă pentru rata de dropout este 0.6, valoare care a fost utilizată și în cadrul acestei implementări. Cu toate acestea, dacă modelul continuă să prezinte fenomenul de supraantrenare (*overfitting*), această valoare ar trebui mărită. Alternativ, dacă tehnica afectează în mod semnificativ predicțiile, rata ar trebui redusă. Există un compromis important care trebuie luat în considerare la implementarea metodei dropout. Din cauza faptului că această tehnică reduce complexitatea modelului, procesul de antrenare poate necesita un număr dublu de epoci pentru a converge către o soluție optimă. Totuși, timpul de antrenare per epocă este redus atunci când se utilizează dropout.

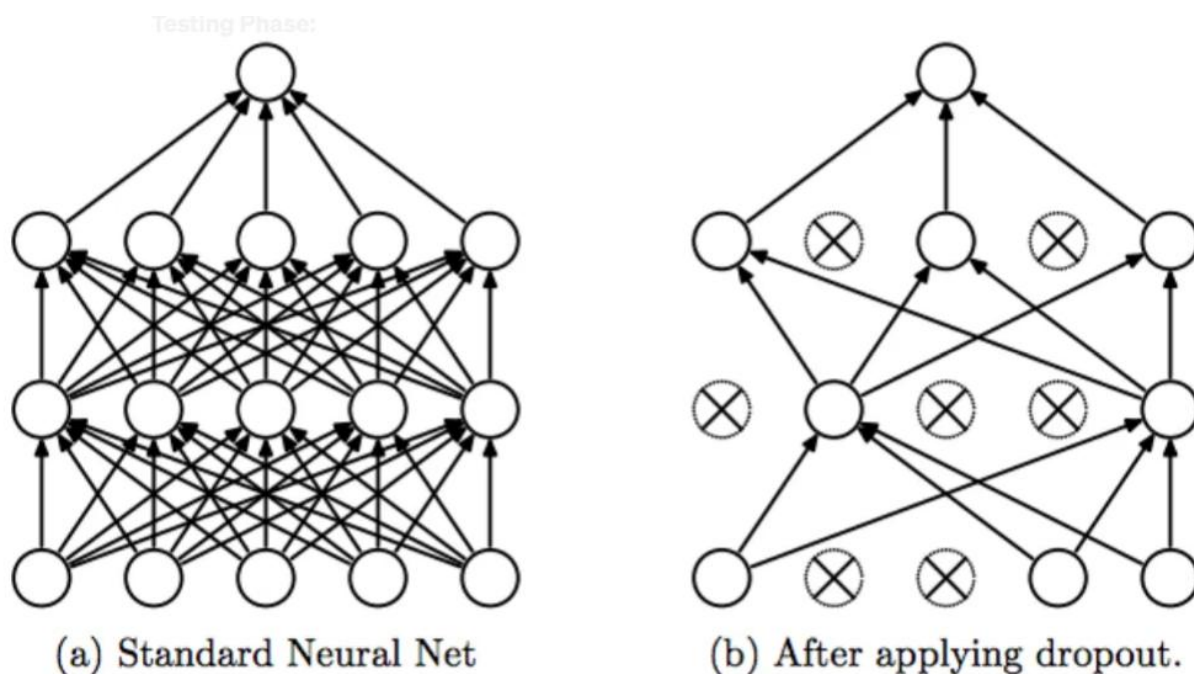


Figure 16 Exemplu de dropout [16]

3.2.6 Învațarea prin transfer (Transfer-Learning)

[23] În dezvoltarea modelelor bazate pe rețele neuronale convoluționale profunde (Deep Convolutional Neural Networks), construirea și antrenarea unei arhitecturi complexe de la zero prezintă bariere majore în mediile medicale. Această abordare tradițională necesită un volum colosal de imagini etichetate riguros și resurse de calcul masive pentru a preveni fenomenul de supraajustare (overfitting). În caz contrar, un număr ridicat de parametri liberi raportat la un set de date limitat determină rețeaua să memoreze zgomotul statistic din imagini în loc să extragă tipare clinice universale valabile.

[24] Pentru a depăși aceste constrângeri și a asigura o capacitate ridicată de generalizare, în cadrul acestui proiect s-a optat pentru utilizarea tehnicii de Învațare prin Transfer (Transfer Learning). Această paradigmă se bazează pe premisa că cunoștințele structurale (cum ar fi detectarea de margini, linii, contraste, forme geometrice și texturi) dobândite de o rețea neuronală în urma antrenării pe un domeniu-sursă masiv pot fi transferate și reutilizate eficient pentru a rezolva o problemă specifică într-un domeniu-țintă restrâns.

În implementarea practică realizată pe platforma Kaggle, procesul de Transfer Learning a fost structurat în trei etape fundamentale:

1. **Selectarea Arhitecturii Preantrenate:** Ca nucleu extractor de caracteristici, a fost selectată rețeaua EfficientNetB0, recunoscută pentru eficiența sa datorită scalării compuse a adâncimii și rezoluției. Modelul a fost încărcat împreună cu ponderile matematice optimizate în prealabil pe setul de date benchmark ImageNet (un set de date gigant, ce cuprinde peste 14 milioane de imagini din diverse clase generale). La import, stratul final de clasificare original al rețelei a fost eliminat (`include_top=False`), izolând doar extractorul de trăsături vizuale.
2. **Înghețarea Parțială a Straturilor:** Pentru a conserva abilitățile fundamentale de procesare vizuală deja optimizate de către Google și pentru a reduce costul computațional, primele 40 de straturi ale rețelei EfficientNetB0 au fost înghețate (`layer.trainable = False`). Acest demers garantează că parametrii structurali de bază rămân imuabili și nu sunt alterați sau distruși în timpul noilor etape de optimizare stohastică.

Prin adoptarea strategiei de Transfer Learning, modelul a beneficiat de 2 avantaje mari. În primul rând, s-a eliminat necesitatea unui set de date de ordinul milioane de imagini medicale. Pe de altă parte, s-a obținut un model cu o amprentă de memorie optimizată și o dinamică a gradientilor

stabilizată, ideal pentru a fi compilat ulterior într-un container Docker și rulat local, pe dispozitivul embedded NVIDIA Jetson Nano.

3.2.7 Platforma de Virtualizare: Docker

[27] Docker este o platformă open-source care permite dezvoltatorilor să construiască, să distribuie, să ruleze, să actualizeze și să gestioneze containere.

Containerele sunt componente standardizate și executabile care combină codul sursă al unei aplicații cu librăriile sistemului de operare (OS) și dependențele necesare pentru a rula acel cod în orice mediu. Este de menționat faptul că, atunci când se discută despre Docker, referirea se face de regulă la Docker Engine, mediul de execuție utilizat pentru construirea și rularea containerelor. De asemenea, termenul „Docker” se poate referi la Docker, Inc., compania care comercializează versiunea comercială a platformei, sau la proiectul open-source Docker, la care contribuie atât Docker, Inc., cât și numeroase alte organizații și persoane fizice.

Containerele simplifică dezvoltarea și livrarea aplicațiilor distribuite. Acestea au devenit din ce în ce mai populare pe măsură ce organizațiile fac tranziția către dezvoltarea nativă în cloud (*cloud-native*) și către medii hibride multicloud. Dezvoltatorii pot crea containere și fără a utiliza Docker, lucrând direct cu funcționalitățile integrate în Linux® și în alte sisteme de operare, însă Docker face procesul de containerizare mult mai rapid și mai facil. La momentul redactării acestui text, Docker a raportat peste 20 de milioane de dezvoltatori care utilizează platforma lunar. La fel ca alte tehnologii de containerizare, inclusiv Kubernetes, Docker joacă un rol crucial în dezvoltarea software modernă, în special în arhitecturile bazate pe microservicii.

3.2.8 Principiul de Funcționare al Containerelor

Existenta containerelor este asigurată de capabilitățile de izolare a proceselor și de virtualizare integrate direct în kernel-ul Linux. Aceste funcționalități includ grupuri de control (*control groups* sau *cgroups*) pentru alocarea resurselor între procese, precum și spații de nume (*namespaces*) pentru restricționarea accesului sau a vizibilității unui proces asupra altor resurse ori zone ale sistemului.

Containerele permit mai multor componente ale unei aplicații să partajeze resursele unei singure instanțe a sistemului de operare gazdă. Această partajare este similară cu modul în care un hipervizor permite mai multor mașini virtuale (VM) să partajeze unitatea centrală de procesare (CPU), memoria și alte resurse ale unui singur server hardware, diferențele structurale fundamentale dintre cele două abordări de virtualizare fiind ilustrate în Figure 17.

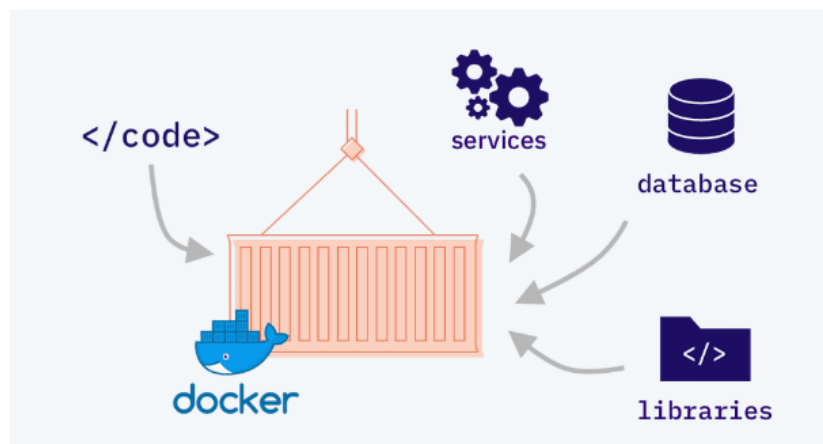


Figure 17 Funcționarea containerelor [17]

3.2.9 Argumente Privind Popularitatea și Utilizarea Docker

Docker este cel mai utilizat instrument de containerizare, deținând o cotă de piață de 82.84%. Platforma este atât de populară în prezent, încât termenii „Docker” și „containere” sunt adesea utilizați ca sinonime. Cu toate acestea, primele tehnologii asociate containerelor au fost disponibile timp de ani sau chiar decenii înainte ca Docker să fie lansat public ca proiect open-source în anul 2013.

În mod special, în anul 2008, tehnologia LinuxContainers (LXC) a fost implementată în kernel-ul Linux, permițând virtualizarea completă pentru o singură instanță de Linux. Deși LXC este utilizat și astăzi, sunt disponibile tehnologii mai noi de kernel Linux. Ubuntu, un sistem de operare Linux modern și open-source, oferă de asemenea această capabilitate. Docker permite dezvoltatorilor să acceseze aceste funcționalități native de containerizare prin utilizarea unor comenzi simple și să le automatizeze prin intermediul unei interfețe de programare a aplicațiilor (API) care reduce volumul de muncă manuală.

3.2.10 Docker pentru proiectul meu

În loc să instalez manual zeci de librării direct în sistemul de operare al plăcii Jetson Nano (ceea ce ar fi putut strica alte configurări sau ar fi generat erori din cauza arhitecturii ARM), eu am rulat un container Docker bazat pe o imagine oficială NVIDIA L4T (Linux for Tegra).

Procesul meu s-a desfășurat astfel:

1. Imaginea de bază: am descărcat de pe internet (din registrul NVIDIA NGC) o imagine Docker preconfigurată special pentru Jetson, numită `nvr.io-nvidia-l4t-tensorflow`.
2. Izolarea: Această imagine a creat un sistem complet izolat (un container) în interiorul plăcii, care conținea deja TensorFlow gata compilat și optimizat să recunoască nucleele GPU (CUDA) ale Jetson-ului.
3. Instalarea în container: în interiorul acestui container Docker securizat, am rulat comanda `apt-get` pentru a instala pachetul `python3-opencv`.
4. Execuția codului live: tot în acest container am rulat scriptul final `live_jetson.py`, iar în acest timp am putut să accesez camera web USB pentru fluxul video live a retinopatiei diabetice.

Capitolul 4

4.1 Rezultate

Următoarea secțiune își propune să demonstreze că toate obiectivele majore ale proiectului au fost îndeplinite cu succes, să discute rezultatele obținute și să ofere o evaluare generală a proiectului.

Pentru început, acuratețea a fost utilizată ca metrică principală de evaluare a performanței diferitelor modele de învățare automată dezvoltate pe parcursul implementării proiectului. Metoda de calcul a acurateții constă în împărțirea numărului de instanțe prezise corect la numărul total de instanțe din setul de date de testare. Setul de date de testare a cuprins un total de 2427 de imagini, reprezentând 15% din întreaga bază de date.

În al doilea rând, după ce au fost adăugate tehnicile de regularizare descrise în secțiunea de implementare cu scopul de a îmbunătăți capacitatea de generalizare a algoritmului pe instanțe noi, nemaiîntâlnite, a început procesul de antrenare al modelului bazat pe arhitectura EfficientNetB0 și funcția de activare Swish. Acesta a fost configurat pentru un ciclu de **50** de epoci pe platforma Kaggle. În urma monitorizării metricilor, s-a observat apariția unui fenomen de plafonare timpurie (early plateau), comportamentul curbelor de învățare și stagnarea performanței fiind ilustrate în Figure 18. Valoarea acurateții pe setul de validare stagnând stabil la pragul de 48% începând cu epoca 28 și menținându-se pe această traiectorie orizontală până la epoca 50.

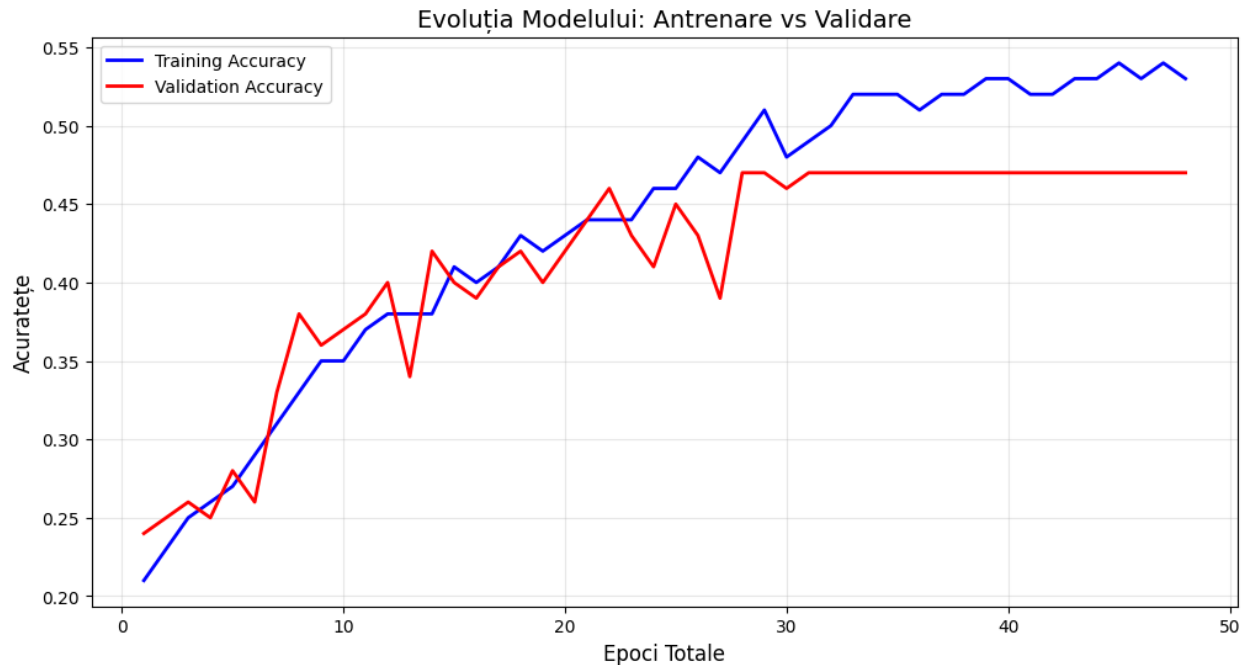


Figure 18 Apariția Vanishing gradient [18]

Acest comportament indică instalarea rigidizării gradientilor (*vanishing gradient*) din cauza saturației asimptotice a funcției Swish în zonele extreme ale straturilor profunde, blocând rețeaua într-un minim local la valoarea menționată de 48% în clasificarea celor 5 stadii diferite de severitate ale retinopatiei diabetice. În plus, modelul de învățare automată a înregistrat o acuratețe de 53 de procente.

În cele din urmă, vor fi analizate rezultatele obținute de modelul implementat prin metoda de învățare prin transfer (*transfer learning*), utilizând arhitectura de ultimă generație cunoscută sub numele de EfficientNetB0. Modelul a reușit să obțină o acuratețe de antrenare de 53% și o acuratețe de testare/validare de 48% în problema de clasificare multclasă (cele 5 stadii) unde se poate vedea în Figure 19.

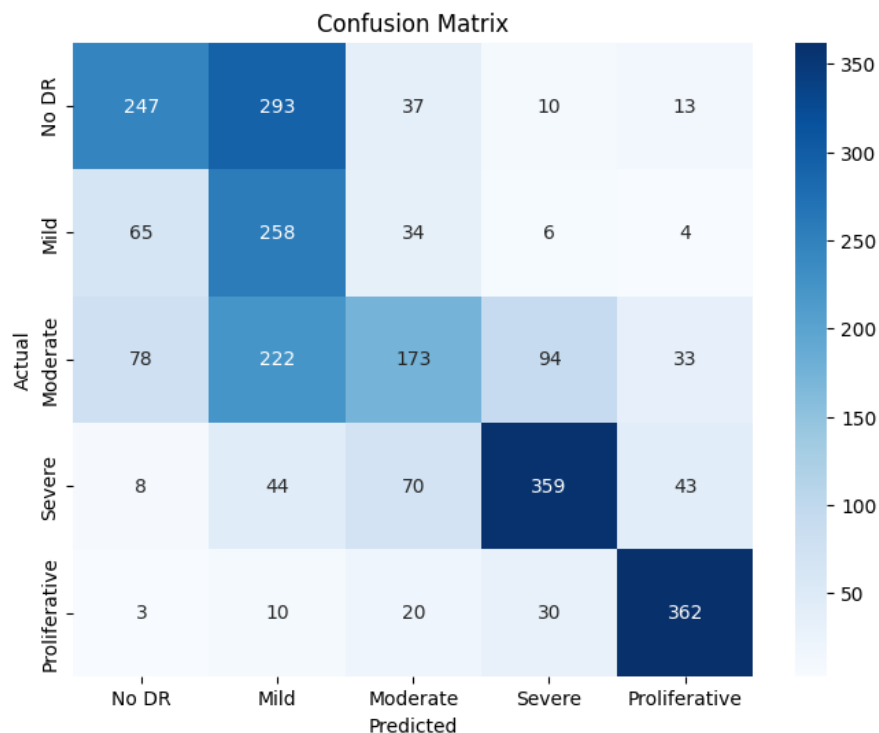


Figure 19 Matricea confuziilor [19]

4.2 Evolutia pierderii (loss)

În primele 29 de epoci, se observă o descreștere constantă și sănătoasă a pierderii de antrenare (*Training Loss* - linia albastră) ce se poate vedea în Figure 20, ceea ce arată că modelul învață. Cu toate acestea, linia roșie (*Validation Loss*) prezintă o anomalie majoră în jurul **epocii 27**, înregistrând un vârf brusc (*spike*) de până la valoarea de ~ 1.68 . La epoca 30 (linia verde), s-a deblocat antrenarea straturilor superioare ale extractorului EfficientNetB0. Intenția algoritmului a fost de a reduce și mai mult eroarea prin adaptarea fină a hărților de caracteristici la patologia retiniană. După epoca 31, comportamentul celor două linii oferă dovada incontestabilă a plafonării și a rigidizării gradientilor (Vanishing Gradient). În timp ce Training Loss (linia albastră) continuă să scadă lent și asimptotic spre valoarea de 1.0 (ceea ce explică de ce acuratețea pe antrenare urcă ușor spre 52%), pierderea pe setul de validare (linia roșie) devine o linie perfect orizontală, rigidizată la valoarea de ~ 1.14 . Toate valorile detaliate în funcție de cele 5 stadii de retinopatie se pot vizualiza în Figure 21.

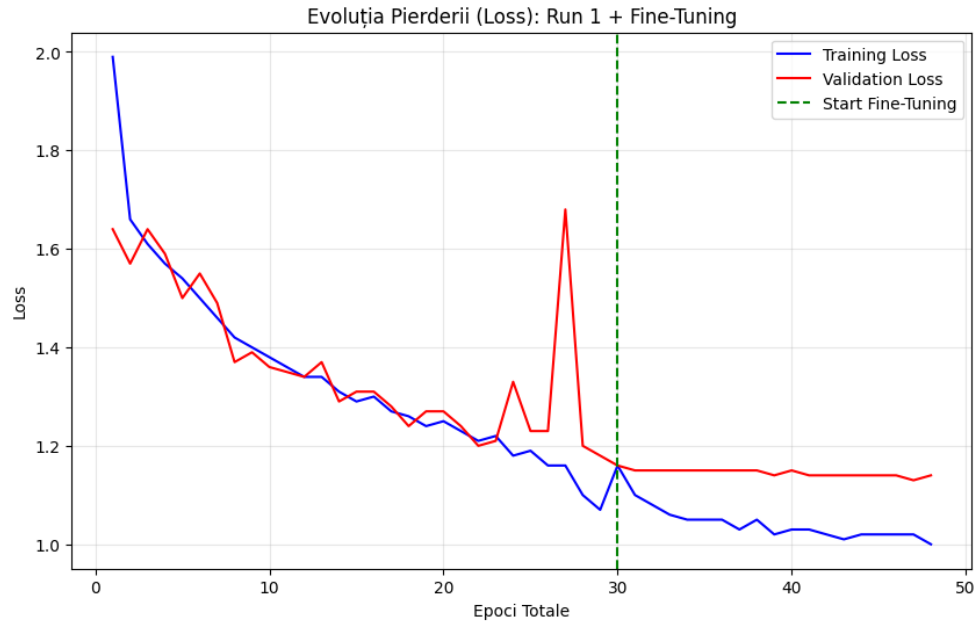


Figure 20 Graficul pentru loss [20]

	precision	recall	f1-score	support
No DR	0.62	0.41	0.49	600
Mild	0.31	0.70	0.43	367
Moderate	0.52	0.29	0.37	600
Severe	0.72	0.69	0.70	524
Proliferative	0.80	0.85	0.82	425
accuracy			0.56	2516
macro avg	0.59	0.59	0.56	2516
weighted avg	0.60	0.56	0.55	2516

Figure 21 Raport cu rezultatele clasificărilor [21]

4.3 Optimizarea pentru un mic boost de performanta

Creșterea performanței modelului de la o acuratețe inițială modestă de 52% la un prag final de 67% în clasificarea celor 5 stadii de severitate ale retinopatiei diabetice nu a fost un rezultat liniar. Pentru a combate rigidizarea gradientilor, au fost reconfigurate patru componente fundamentale ale arhitecturii: pipeline-ul de preprocesare, rata de abandon (*dropout rate*), dimensiunea loturilor de date (*batch size*) și rata de învățare inițială (*learning rate*).

4.4 Tranziția de la Preprocesarea Standard la o accentuare a detaliilor

Într-o primă fază a proiectului, s-a optat pentru o metodă de preprocesare simplă, specifică unei abordări convenționale de început în viziunea artificială. Aceasta consta exclusiv în redimensionarea imaginilor brute la 320x320 pixeli și normalizarea valorilor acestora în intervalul [0,1] prin împărțirea la 255.0 (operație realizată direct pe pixelii RGB). Deși intuitivă, această abordare nu oferea performanțe înalte. Imaginile de fund de ochi sufereau din cauza variațiilor extreme de iluminare globală provocate de aparatura medicală, iar detaliile clinice esențiale (cum ar fi micile microanevrisme sau exsudatele punctiforme) se pierdeau în textura de fundal a retinei. Din această cauză, extractorul de caracteristici nu găsea tipare discriminative, iar acuratețea s-a blocat la 52%.

Rezolvarea a constat în înlocuirea completă a acelei metode cu un algoritm dedicat de accentuare a detaliilor de înaltă frecvență, implementat prin funcția `accentuare_detalii`. Algoritmul acționează la nivel matematic prin extragerea unei măști de frecvență joasă.

$$I_{acc} = (I_{original} - I_{blur}) \cdot \alpha + 128 \quad (1.1)$$

4.5 Reducerea dimensiunii de batch (batch size)

O altă modificare cu impact direct asupra dinamicii antrenării a fost reducerea dimensiunii lotului de imagini procesate simultan de la standardul de 32 la 16. În contextul utilizării unei rezoluții mari de 320x320 pixeli, un lot de 32 de imagini solicita memoria cache a GPU-ului, ducând la timpi mari de așteptare per epocă din cauza transferului dens de date. Trecerea la un batch de 16 a fluidizat considerabil modelul, iar modificările la graph se pot vedea în Figure 22, reducând vizibil timpul de antrenare per epocă pe platforma Kaggle.

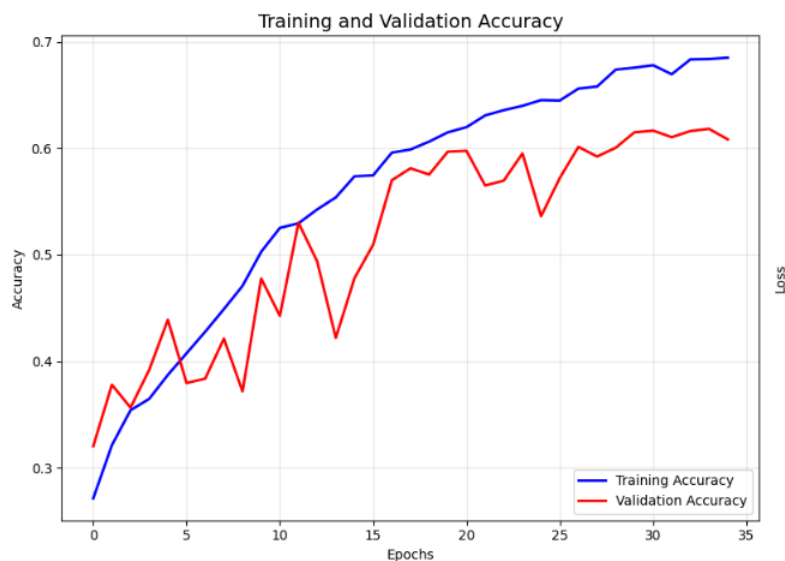


Figure 22 Graph-ul pentru rezultatele de test + train dupa optimizare [22]

4.6 Ajustarea a ratei de dropout de la 0.4 la 0.6

Pentru a asigura stabilitatea structurală a stratului complet conectat (*Dense*) de 512 neuroni și pentru a preveni fenomenul de supraantrenare (*overfitting*), s-a decis modificarea hiperparametrului de regularizare. În fazele inițiale, s-a utilizat o rată standard de Dropout(0.4). Totuși, din cauza complexității mari a rețelei EfficientNetB0, modelul manifesta tendința de a memora zgomotul din datasetul de antrenare.

Prin creșterea a ratei la Dropout(0.6), s-a impus ca 60% din conexiunile stratului intermediar să fie dezactivate aleatoriu la fiecare pas de propagare. Această constrângere severă a eliminat co-adaptarea neuronilor și a forțat rețeaua să dezvolte reprezentări interne independente și mult mai robuste.

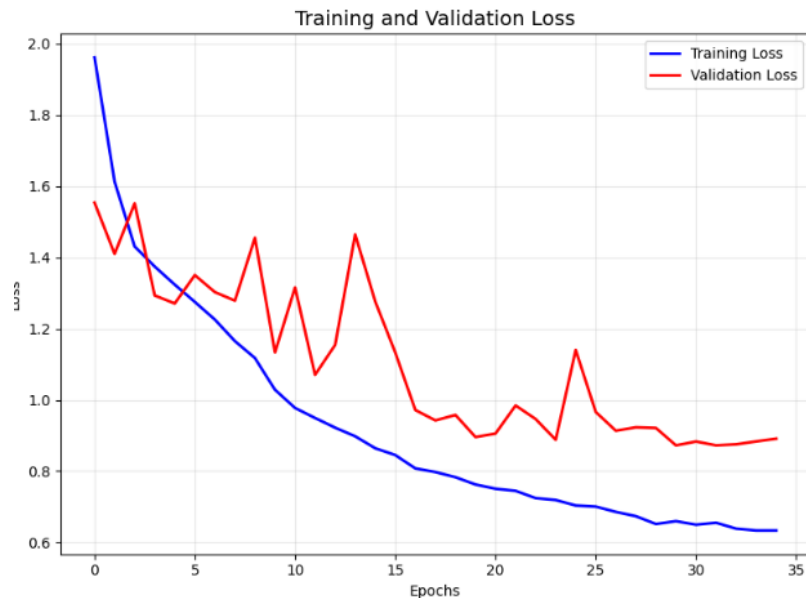


Figure 23 Graph-ul pentru rezultatele de loss [23]

	precision	recall	f1-score	support
No DR (0)	0.58	0.64	0.61	675
Mild (1)	0.33	0.45	0.38	366
Moderate (2)	0.62	0.37	0.46	675
Severe (3)	0.88	0.93	0.91	393
Proliferative (4)	0.88	0.99	0.93	319
accuracy			0.62	2428
macro avg	0.65	0.67	0.65	2428
weighted avg	0.63	0.61	0.61	2428

Figure 24 Raport cu rezultatele clasificărilor optimizate [24]

4.7 Identificarea punctului optim pentru rata de învățare

Calibrarea ratei de învățare (*learning rate*) a reprezentat una dintre cele mai mari provocări matematice ale proiectului. Inițial, s-au testat valori standard, mai mari (cum ar fi 3×10^{-3} sau 3×10^{-4}). Aceste rate mari provocau modificări violente ale ponderilor în primele epoci, împingând activările neuronilor direct în zonele de saturare asimptotică ale funcției Swish, declanșând instantaneu fenomenul de vanishing gradient și blocând rețeaua.

După 2-3 experimente succesive de reducere și monitorizare a curbelor de *Loss*, s-a descoperit experimental că punctul optim pentru acest proiect este de 5×10^{-5} (`learning_rate=5e-`

5). Această rată de învățare extrem de fină și conservatoare, corelată cu funcția `ReduceLROnPlateau` (care înjumătățește valoarea dacă progresul stagnează), a permis modelului să execute pași extrem de preciși în peisajul matematic complex. Această calibrare fină a menținut gradientii activi, deblocând modelul din stadiul stagnat de 52% și permițându-i să își maximizeze capacitatea de diagnosticare până la performanța finală de 67% acuratețe.

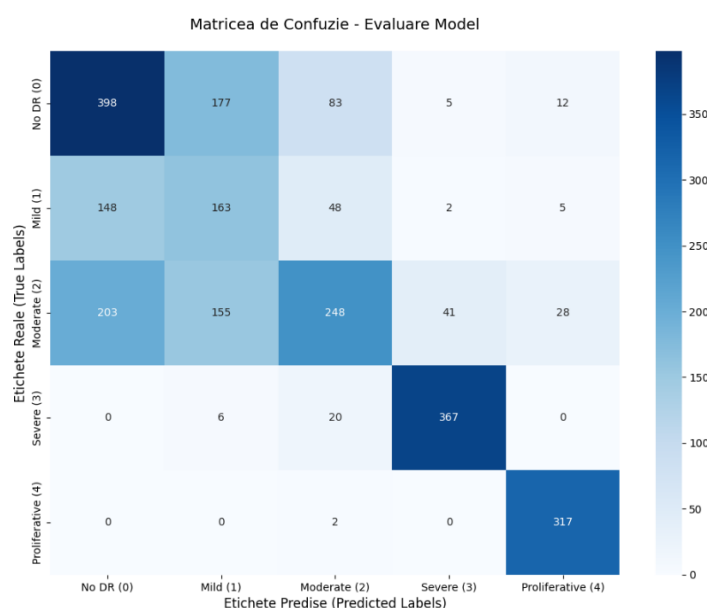


Figure 25 Matricea confuziilor pentru modelul optimizat [25]

4.8 Testarea directă pe placuța Jetson Nano

Pentru a porni placa Jetson Nano și a rula sistemul de screening retinal live, configurarea hardware trebuie realizată într-o anumită ordine, astfel încât vom avea o alimentare stabilă pentru a preveni opririle bruște în timpul rulării programului.

1. Conectarea perifericelor: înainte de a introduce cablul de alimentare, se conectează toate perifericele necesare interfeței grafice și rulării scriptului (mouse, tastatură, camera web)
2. Conectarea alimentării: este necesar să folosim un încărcător care suportă 5V cu 3A adică max 15W deoarece când placa este solicitată consumul de curent crește instantaneu
3. Momentul de pornire și logare: atunci când se conectează cablul de alimentare care trebuie să fie de minim 3A, placa va boota automat, ea neavând un buton de start, se va aprinde automat după ce detectează curent un buton LED verde pe placa.
4. Intrarea în proiect: Odată intrat în placuță trebuie să deschidă terminalul care este localizat pe Desktop și se va naviga în folderul unde este proiectul. Odată intrat în folderul

proiectului se va rula comanda de intrare in Docker care totodata activeaza si camera iar ulterior, in interiorul containerului se va lansa interviata live.

4.9 Rezultatele testelor direct pe plăcuța Jetson Nano

Deoarece sistemul utilizează o cameră web USB standard ca senzor de achiziție, s-au făcut verificările utilizând un smartphone, pe care au fost puse imagini diferite față de setul de date folosit pentru antrenare. Telefonul a fost poziționat drept în fața camerei, la o distanță optimă pentru a asigura focalizarea corectă și încadrarea completă a ochiului în zona de interes a interfeței grafice.

În cadrul acestui proces, s-a constatat că controlul iluminării ambientale este un factor critic. O lumină ambientală prea puternică sau prezența unor surse directe de lumină în încăpere reflectă pe ecranul telefonului, creând reflexii parazite sau artefacte de saturație a albului. Aceste distorsiuni optice pot masca detaliile de finețe din textură (cum ar fi microanevrismele sau exsudatele) și pot induce erori în procesul de clasificare, scăzând acuratețea globală. Prin urmare, pentru o performanță optimă, testarea s-a efectuat într-un mediu cu iluminare scăzută, permițând algoritmului de accentuare să facă preziceri cu o acuratețe ridicată.

În primul scenariu, modelul a analizat o rețină fără semne clinice de boală. Rețeaua neuronală prelucrează cadrul live, aplică masca de frecvență înaltă pentru a netezi fundalul și identifică corect absența leziunilor. După cum se poate observa în Figure 26, sistemul afișează în timp real pe ecran clasa „No DR (0)”.

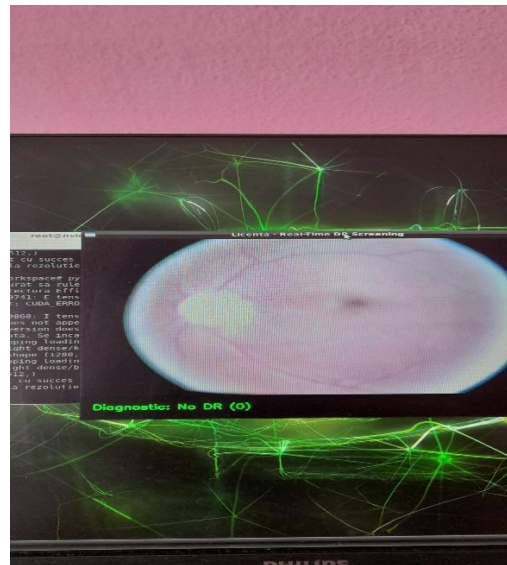


Figure 26 Rezultatul NoDr din aplicatie direct de pe placa JetsonNano [26]

În cel de-al doilea scenariu, sistemul a fost pus în fața unei imagini retiniene ce prezenta multiple hemoragii punctiforme, microanevrisme dense și exsudate dure extinse. Datorită filtrului de accentuare locală care a mărit contrastul acestor detalii modelul de machine learning a clasificat aceasta poza ca fiind “Severe (3)” unde se poate vizualiza în Figure 27.



Figure 27 Rezultatul Severe din aplicație direct de pe placa Jetson Nano [27]

În cel de-al treilea scenariu, sistemul a analizat o imagine retinală aflată într-o fază avansată de evoluție a patologiei, caracterizată prin prezența neovascularizației (formarea de noi vase de sânge fragile), a hemoragiilor preretiniene masive și a proliferării de țesut fibros. Algoritmul a procesat eficient structurile vasculare anormale evidențiate de etapa de filtrare și a clasificat instanța live în clasa de severitate maximă, respectiv „*Proliferative DR (4)*”, interfața aplicației în acest punct fiind redată în Figure 28.

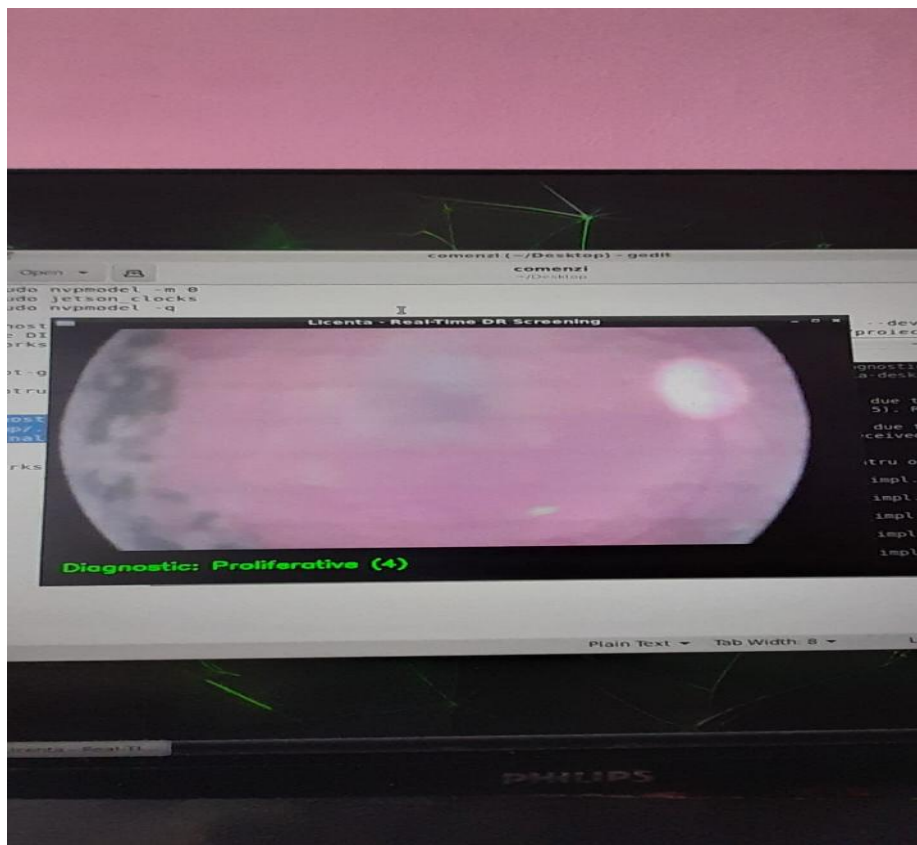


Figure 28 Rezultatul Proliferative din aplicație direct de pe placa Jetson Nano [28]

În cel de-al patrulea scenariu, modelul de *machine learning* a primit ca intrare o imagine de fund de ochi ce prezenta un stadiu intermediar al afecțiunii. Aceasta conținea mai multe microanevrisme, hemoragii retiniene moderate și exsudate dure, însă fără a îndeplini criteriile stricte pentru stadiul sever (regula 4-2-1). În urma aplicării măștilor de contrast, rețeaua neuronală a identificat corect distribuția leziunilor și a atribuit imaginii clasa „*Moderate DR (2)*”, confirmarea vizuală a predicției fiind ilustrată în Figure 29.



Figure 29 Rezultatul Moderate direct din aplicația Jetson Nano [29]

4.10 Dezavantajele proiectului

Primul dezavantaj major identificat în urma testelor de sistem vizează rigiditatea pipeline-ului de captură și analiză dinamică a cadrelor. Deși interfața grafică este complet nativă, sistemul manifestă o incapacitate de re-inițializare automată a stării de clasificare la schimbarea dinamică a contextului de intrare. Mai precis, în momentul în care o imagine a fundului de ochi este plasată în fața camerei, modelul o analizează, filtrele matematice extrag caracteristicile patologice, iar stratul final Softmax generează distribuția probabilistică și afișează clasa corectă pe ecran. Cu toate acestea, dacă persoana decide să schimbe imaginea curentă cu o alta, prezentând un nou fund de ochi în câmpul vizual al camerei fără a opri execuția, rețeaua neuronală tinde să rămână blocată în starea de clasificare precedentă sau să ofere predicții eronate induse de persistența buffer-ului video anterior.

Această problemă tehnică este strâns legată de modul în care este gestionată memoria buffer-ului de captură și de latența asincronă dintre thread-ul responsabil de interfața grafică și cel dedicat inferenței modelului EfficientNetB0. În lipsa unui algoritm implementat pentru detectarea automată a schimbării scenei (cum ar fi o analiză pe baza histogramei cadrelor consecutive),

buffer-ul camerei acumulează cadre de tranziție, care corup procesul de calcul al gradientului local în straturile convoluționale. În stadiul actual al aplicației, singura metodă sigură prin care operatorul poate solicita o nouă clasificare validă pentru un pacient diferit este repornirea completă a aplicației. Acest ciclu de oprire și restart manual întrerupe continuitatea fluxului medical, crește timpul de așteptare per investigație și diminuează considerabil experiența generală de utilizare într-un mediu clinic real, unde eficiența timpului este critică.

Al doilea dezavantaj, de natură strict computațională și optică, se referă la vulnerabilitatea rețelei neuronale în fața variațiilor de luminozitate din mediul în care se efectuează achiziția de imagini. Modelele de învățare profundă antrenate pe seturi de date medicale standardizate (cum sunt cele de pe Kaggle) devin extrem de sensibile la factorii externi atunci când sunt transpuse într-un scenariu de achiziție live prin intermediul unei camere fizice. În cazul de față, măștile de frecvență înaltă și filtrele de accentuare locală implementate în etapa de preprocesare au rolul de a măări contrastul structurilor fine din retină, cum ar fi microanevrismele punctiforme și exsudatele dure. Din păcate, aceste filtre matematice nu fac distincție între elementele anatomice reale și artefactele optice cauzate de mediu.

Orice rază de lumină ambientală care cade neuniform pe suprafața imaginii analizate, orice reflexie parazită pe lentila camerei sau fluctuație a intensității luminoase din încăperea este preluată instantaneu de algoritmul de filtrare. Filtrul va interpreta aceste reflexii și puncte suprasaturate de lumină albă drept modificări structurale majore, transformându-le în zgomot vizual de intensitate mare. Când aceste imagini alterate ajung în straturile convoluționale profunde, rețeaua neuronală este indusă în eroare, generând clasificări complet eronate (de exemplu, un ochi perfect sănătos, de clasa 0, poate fi clasificat ca fiind în stadiu moderat sau sever din cauza unor pete de lumină interpretate greșit ca leziuni).

Din acest motiv, operarea sistemului în condiții ideale impune o constrângere ambientală severă: aplicația necesită utilizarea într-un mediu cu o luminozitate controlată și redusă (semi-întuneric). Recomandarea de a diminua lumina ambientală este esențială pentru a atenua apariția reflexiilor pe ecrane sau pe hărțile imprimare, asigurând o consistență a pixelilor de intrare. Această dependență critică de factorii de mediu reduce robustețea algoritmului, făcându-l dificil de utilizat în cabinete medicale luminate normal sau în condiții de screening rapid unde lumina nu poate fi ajustată cu ușurință.

Capitolul 5

5.1 Posibile optimizări și direcții viitoare de dezvoltare

Deși prototipul actual implementat pe platforma NVIDIA Jetson Nano oferă performanțe satisfăcătoare pentru un screening în timp real, sistemul poate fi optimizat în continuare prin tehnici avansate de compilare și accelerare hardware.

Cea mai importantă direcție de optimizare o reprezintă conversia modelului din formatul nativ TensorFlow (.h5) în formatul NVIDIA TensorRT.

TensorRT este un motor de inferență de înaltă performanță, conceput special de NVIDIA pentru a optimiza rețelele neuronale pe cipuri grafice embedded. Printr-o astfel de conversie, s-ar fi obținut următoarele avantaje critice pentru proiect:

1. Maximizarea utilizării plăcii video (GPU): Deși TensorFlow poate rula pe GPU, TensorRT restructurează straturile rețelei EfficientNetB0 special pentru arhitectura hardware Maxwell a plăcuței Jetson Nano. Acest lucru elimină blocajele de memorie și optimizează nucleele CUDA.
2. Fuziunea straturilor (Layer Fusion): TensorRT combină straturile matematice ale rețelei (cum ar fi operațiile de convoluție, activare și normalizare) într-o singură operație optimizată. Astfel, se reduce drastic numărul de treceri prin memorie, ceea ce duce la o viteză mult mai mare de procesare.
3. Cuantizarea preciziei (FP16): Modelul actual rulează în precizie standard pe 32 de biți (FP32). Prin TensorRT, modelul poate fi convertit la precizie semi-plutitoare pe 16 biți (FP16). Această optimizare reduce aproape la jumătate memoria RAM ocupată și poate dubla numărul de cadre pe secundă (FPS), fără a pierde din acuratețea diagnosticării retinopatiei.

Pe scurt, transformarea modelului în TensorRT ar fi pasul următor cel mai inteligent. Această schimbare ar transforma prototipul actual într-un aparat medical real și foarte rapid, capabil să analizeze imaginile video instantaneu, fără nicio întrerupere sau sacadare.

5.2 Concluzii

În concluzie, studiul de față a reușit să își atingă obiectivul de a utiliza diverse tehnici de învățare automată (machine learning) pentru a clasifica stadiile de severitate ale retinopatiei diabetice. Modelele de învățare profundă au performat cu un scor de acuratețe de 67% în clasificarea celor cinci stadii de severitate ale retinopatiei diabetice: fără retinopatie diabetică, stadiu incipient (mild), stadiu moderat, stadiu sever și retinopatie diabetică proliferativă.

Pe parcursul implementării proiectului au fost dezvoltate și antrenate două configurații bazate pe învățare profundă și rețele neuronale convoluționale. Prima abordare a constatat într-un model simplu, utilizat ca punct de referință pentru a evalua comportamentul rețelei pe imaginile retiniene preprocesate standard. Cea de-a doua abordare reprezintă modelul optimizat, în cadrul căruia au fost integrate tehnici avansate de augmentare și algoritmul de accentuare a contrastului. Ambele metode sunt introduse și descrise în detaliu, oferind informațiile esențiale privind evoluția performanței.

Rezultatele au arătat că, prin adăugarea acestui filtru, modelul EfficientNetB0 a devenit mult mai precis decât modelul simplu de la care am pornit. În plus, cea mai importantă realizare a proiectului este că nu s-a oprit doar la niște teste pe calculator. Am reușit să mutăm acest model inteligent pe o plăcuță portabilă NVIDIA Jetson Nano, rulat în interiorul unui container Docker. Acest lucru permite sistemului să analizeze imaginile pe loc, în timp real, direct de la o cameră web, fără să aibă nevoie de internet. Este o soluție ieftină și portabilă, ideală pentru a fi folosită ca un prim test rapid în orice cabinet medical.

Privind în ansamblu traiectoria întregului proiect, această lucrare demonstrează că metodele moderne de învățare profundă pot constitui un instrument valoros de suport în screening-ul oftalmologic, chiar și în condițiile unor resurse computaționale limitate. Punctul de plecare al cercetării a fost o problemă medicală cu impact social real: retinopatia diabetică reprezintă una dintre principalele cauze de orbire evitabilă la nivel global, afectând milioane de pacienți care, în lipsa unui diagnostic precoce, pierd oportunitatea unui tratament eficient. Prin prisma acestui context, orice sistem capabil să automatizeze și să accelereze procesul de triaj vizual constituie o contribuție concretă la sănătatea publică.

Din punct de vedere tehnic, una dintre lecțiile fundamentale ale acestui proiect a fost importanța critică a etapei de preprocesare a datelor. Imaginile de fund de ochi prezintă variații semnificative de iluminare, contrast și claritate, generate de diversitatea aparaturii medicale utilizate la captură. Simpla redimensionare și normalizare a valorilor de pixel, deși intuitivă ca primă abordare, s-a dovedit insuficientă pentru a extrage tiparele clinice relevante microanevrisme, exsudate dure sau hemoragii retiniene punctiforme. Introducerea algoritmului de accentuare a detaliilor de înaltă frecvență, bazat pe scăderea componentei de frecvență joasă (blur Gaussian)

din imaginea originală, a reprezentat schimbarea fundamentală care a deblocat performanța modelului, permițând saltul de la 52% la 67% acuratețe pe setul de validare.

Procesul de optimizare a hiperparametrilor a evidențiat, de asemenea, fragilitatea antrenării rețelelor neuronale profunde. Alegerea greșită a ratei de învățare a generat fenomenul de *vanishing gradient*, blocând modelul la o acuratețe de 48% încă din primele epoci. Calibrarea fină la valoarea de 5×10^{-5} , combinată cu callback-ul ReduceLROnPlateau și cu reducerea dimensiunii lotului de la 32 la 16 imagini, a permis traversarea acestui obstacol și optimizarea continuă a ponderilor. Aceste experiențe confirmă că, în cazul modelelor profunde aplicate pe imagini medicale, ajustarea metodică și iterativă a hiperparametrilor nu este opțională, ci o etapă indispensabilă a procesului de dezvoltare.

Implementarea pe plăcuța NVIDIA Jetson Nano reprezintă poate cea mai valoroasă dimensiune a acestui proiect din perspectiva aplicabilității practice. Transpunerea unui model antrenat în mediul cloud (platforma Kaggle) pe un dispozitiv embedded cu resurse limitate implică o serie de provocări tehnice suplimentare: incompatibilități de versiuni între TensorFlow și arhitectura ARM, gestionarea dependențelor de librării, configurarea accesului la camera USB și optimizarea fluxului de inferență. Soluția adoptată containerizarea întregului mediu de execuție prin Docker, pe baza unei imagini oficiale NVIDIA L4T preconfigurată pentru Jetson s-a dovedit robustă și reproductibilă, izolând aplicația de variabilele sistemului gazdă și asigurând portabilitatea deploymentului.

Testele de inferență realizate în timp real au confirmat funcționalitatea sistemului: modelul a clasificat corect atât retine sănătoase (clasa „No DR”), cât și retine cu semne de patologie avansată (stadiile „Severe” și „Proliferative DR”), utilizând ca senzor o cameră web USB standard.

În ansamblu, acest proiect demonstrează viabilitatea unui sistem de screening retinal bazat pe inteligență artificială, portabil și accesibil din punct de vedere al costurilor hardware. Deși acuratețea de 67% atinsă în clasificarea celor cinci clase nu satisface încă standardele clinice necesare utilizării autonome în practica medicală, rezultatul este remarcabil în contextul constrângerilor impuse: un set de date public dezechilibrat, resurse de antrenare limitate la platforma Kaggle și un dispozitiv de inferență embedded cu putere de calcul redusă. Acest prototip funcțional poate constitui o bază solidă pentru cercetări ulterioare orientate spre creșterea acurateții, optimizarea vitezei de inferență prin TensorRT și extinderea capacităților de achiziție optică.

Bibliografie

- [17] <https://accesto.com/blog/what-is-docker-and-why-to-use-it/>
- <https://www.geeksforgeeks.org/devops/introduction-to-docker/>
- [16] <https://medium.com/@amarbudhiraja/https-medium-com-amarbudhiraja-learning-less-to-learn-better-dropout-in-deep-machine-learning-74334da4bfc5>
- [18] <https://www.proquest.com/openview/e275e76fe0a3d35b617f0437b905b46c/1?pq-origsite=gscholar&cbl=51922&diss=y>
- [19] https://link.springer.com/chapter/10.1007/978-3-030-85292-4_3
- [29] https://ieeexplore.ieee.org/abstract/document/9336290?casa_token=jyBXEHF-wiYAAAAA:ee-YzFNea065gjaow8fn1lJ9K1--BFPN2BGYmGQo7y714JHUnbnnqUblNrbwRtiJy3aCidkLGrZV
- [30] <https://pubs.acs.org/doi/full/10.1021/ci0342472>
- [20] <https://developer.nvidia.com/embedded/learn/jetson-nano-2gb-devkit-user-guide>
- [21] https://link.springer.com/chapter/10.1007/978-981-99-0412-9_46
- [22] <https://lutpub.lut.fi/handle/10024/169800>
- [24] https://link.springer.com/chapter/10.1007/978-3-030-89010-0_4
- [14] <https://encord.com/blog/an-introduction-to-cross-entropy-loss-functions/>
- [1] <https://www.kaggle.com/code/randaelnagar/diabetic-retinopathy-using-swin>
- [2] <https://link.springer.com/article/10.1007/s11042-020-10238-4>
- [29] https://link.springer.com/chapter/10.1007/978-981-99-6706-3_27
- [4] <https://link.springer.com/article/10.1007/s12652-020-01901-7>
- [5] https://www.researchgate.net/figure/A-basic-skeleton-of-a-convolutional-neural-network-CNN-converts-the-image-to-a-form-good_fig3_346483632
- [6] https://www.researchgate.net/figure/mpact-of-learning-rate-on-training-32_fig5_337940311
- [7] <https://medium.com/swlh/machine-learning-how-to-prevent-overfitting-fdf759cc00a9>
- [8] <https://labelyourdata.com/articles/data-augmentation>

- [9] https://www.researchgate.net/figure/Inception-module-3-three-convolutional-1-1-3-3-5-5-and-a-3-3-max-pooling_fig9_324546396
- [13] https://www.ebay.co.uk/itm/363731902373?chn=ps&_ul=GB&google_free_listing_action=view_item
- [28] <https://link.springer.com/article/10.1186/s40537-021-00444-8>
- [3] <https://medium.com/@soumallya160/the-ultimate-deep-dive-into-convolution-neural-network-993c27c84d78>
- [10] <https://medium.com/appyhigh-technology-blog/convolutional-neural-networks-a-brief-history-of-their-evolution-ee3405568597>
- [15] <https://codingnomads.com/validation-learning-curve-overfitting-and-underfitting>
- [23] <https://link.springer.com/article/10.1186/S40537-016-0043-6>
- [25] <https://dl.acm.org/doi/full/10.1145/3510413>
- [26] <https://www.mdpi.com/2079-9292/10/20/2470>
- [27] <https://www.ibm.com/think/topics/docker>

Anexa

Codul pentru antrenarea modelului:

Codul pentru machine learning 1

```
import pandas as pd
import numpy as np
import os
import glob
import matplotlib.pyplot as plt
import cv2
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.applications import EfficientNetB0
import os
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
df_labels = pd.read_csv("/kaggle/input/datasets/tanlikesmath/diabetic-retinopathy-
resized/trainLabels.csv")
#print(df_labels.head(7))
df_labels_cleared = df_labels[['image', 'level']]
#print(df_labels_cleared.shape) -- am realizat faptul ca acest index este invizibil pentru
model.
#print(df_labels_cleared)

path = "/kaggle/input/datasets/tanlikesmath/diabetic-retinopathy-
resized/resized_train/resized_train"
files = glob.glob(os.path.join(path, "**/*.jpeg"), recursive=True)
file_count = len(files)

#print(file_count) -- avem 35108 imagini pentru antrenare
#print(df_labels_cleared.groupby("level").size())
# 0 - No DR -25802-
# 1 - Mild -2438-
# 2 - Moderate -5288-
# 3 - Severe -872-
# 4 - Proliferative DR -708-
# plt.figure(figsize=(15,20))

# for i in range(5):
#     referinta = df_labels_cleared[df_labels_cleared["level"] == i].iloc[0]
```

```

#     img_name = str(referinta["image"])
#     if not img_name.lower().endswith(".jpeg"):
#         img_name += ".jpeg"
#     img_path = os.path.join(path, img_name)
#     img = cv2.imread(img_path)
#     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
#     #img_blur = cv2.addWeighted(img, 4, cv2.GaussianBlur(img, (0,0), 10), -4, 128)

#     plt.subplot(1,5,i+1)
#     plt.imshow(img)
#     plt.title(f"Level {i}")
#     plt.axis("off")

# plt.show()
# print(referinta)

file_paths = []

for image_name in df_labels_cleared["image"]:
    name_with_ext = str(image_name) + ".jpeg"
    file_paths.append(os.path.join(path, name_with_ext))

df_labels_cleared["file_paths"] = file_paths

#print(df_labels_cleared.head(6))
#print(df_labels_cleared["file_paths"])
# print(len(x_train),len(y_train), len(x_test), len(y_test))
#print(class_weight_dict)

def accentuare_detalii(img, alfa = 4, sigma=10):
    img_float = img.astype(np.float32)
    Ib = cv2.GaussianBlur(img_float, (0, 0), sigma)
    I_acc = (img_float - Ib) * alfa + 128
    I_acc = np.clip(I_acc, 0, 255).astype(np.uint8)
    return I_acc

# def procesare_imagine(file_path):
#     img = cv2.imread(file_path)
#     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
#     img = cv2.resize(img, (300,300))
#     alfa = 4
#     sigma = 10
#     Ib = cv2.GaussianBlur(img, (0, 0), sigma)
#     img = cv2.addWeighted(img, alfa, Ib, -alfa, 128)
#     img = img.astype(np.float32) / 255.0
#     # clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
#     # img[:, :,0] = clahe.apply(img[:, :,0])
#     # img[:, :,1] = clahe.apply(img[:, :,1])
#     # img[:, :,2] = clahe.apply(img[:, :,2])
#     # img = img / 255.0
#     return img

```

```

# test_img = procesare_imagine("/kaggle/input/datasets/tanlikesmath/diabetic-retinopathy-
resized/resized_train/resized_train/81_right.jpeg")

#print(test_img.shape, test_img, test_img.min(), test_img.max())    #224/224 3-RGB iar
pixelii sunt de la 0 care este pixel negru pana la 0.964 care este aproape de pixel 255 -
alb

df_balansat_700 = pd.DataFrame()

# for level in range(0,5):
#     df_level = df_labels_cleared[df_labels_cleared["level"] == level]
#     numere = min(len(df_level), 3000)
#     df_sample = df_level.sample(n=numere, random_state=41)
#     df_balansat_700 = pd.concat([df_balansat_700, df_sample])
# df_balansat_700 = df_balansat_700.reset_index(drop=True)

for level in [0, 1, 2]:
    df_level = df_labels_cleared[df_labels_cleared["level"] == level]
    df_sample = df_level.sample(n=min(len(df_level), 4500), random_state=41)
    df_balansat_700 = pd.concat([df_balansat_700, df_sample])

for level in [3, 4]:
    df_level = df_labels_cleared[df_labels_cleared["level"] == level]
    df_multiplied = pd.concat([df_level] * 3)
    df_balansat_700 = pd.concat([df_balansat_700, df_multiplied])

df_balansat_700 = df_balansat_700.sample(frac=1).reset_index(drop=True)

#print(df_balansat_700.groupby("level").size())
#print(df_balansat_700.head(7))
# 1500 - 0; 1500 - 1; 1500 - 2; 872 - 3; 708 - 4

x_train, x_test, y_train, y_test = train_test_split(df_balansat_700["file_paths"],
df_balansat_700["level"], test_size=0.15, stratify=df_balansat_700["level"], random_state=41)

def load_and_preprocess(path, label):
    img = tf.io.read_file(path)
    img = tf.image.decode_jpeg(img, channels=3)
    img = tf.image.resize(img, [320, 320])
    img = tf.cast(img, tf.float32)
    blur = tf.nn.avg_pool2d(tf.expand_dims(img, 0), ksize=11, strides=1, padding='SAME')
    blur = tf.squeeze(blur, 0)
    img = (img - blur) * 4.0 + 128.0
    img = tf.clip_by_value(img, 0.0, 255.0) / 255.0
    return img, label
    # img = tf.add(tf.multiply(img, 4.0), tf.multiply(blur, -4.0))
    # img = tf.add(img, 128.0)
    # img = tf.clip_by_value(img, 0.0, 255.0) / 255.0
    # return img, label

#print(f"Avem {len(x_train)} imagini de train si {len(x_test)} imagini de test")

```

```

clase = np.asarray([0,1,2,3,4])
weight = compute_class_weight(class_weight="balanced", classes=clase, y=y_train)
class_weight_dict = dict(zip(clase,weight))

#print(len(x_train), len(x_test))
# x_train_list = []
# for filepath in x_train:
#     img = procesare_image(filepath)
#     x_train_list.append(img)
# x_train_list = np.array(x_train_list)

# x_test_list = []
# for filepath in x_test:
#     img = procesare_image(filepath)
#     x_test_list.append(img)
# x_test_list = np.array(x_test_list)

train_ds = tf.data.Dataset.from_tensor_slices((list(x_train), list(y_train)))
train_ds = train_ds.shuffle(len(x_train)).map(load_and_preprocess,
num_parallel_calls=tf.data.AUTOTUNE).batch(16).prefetch(tf.data.AUTOTUNE)

test_ds = tf.data.Dataset.from_tensor_slices((list(x_test), list(y_test)))
test_ds = test_ds.map(load_and_preprocess,
num_parallel_calls=tf.data.AUTOTUNE).batch(16).prefetch(tf.data.AUTOTUNE)

y_train = np.array(y_train)
y_test = np.array(y_test)

#print(test_img.min(), test_img.max())
#print(y_test.shape, y_train.shape)
# am facut si datele ce nu le stie adica labels ca array pentru a putea face comparatia
#print(x_train_list.shape)

base_model = tf.keras.applications.EfficientNetB0(
    weights="imagenet",
    include_top=False,
    input_shape=(320, 320, 3)
)

base_model.trainable = True

for layer in base_model.layers[:40]:
    layer.trainable = False

inputs = tf.keras.Input(shape=(320, 320, 3))
x = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal_and_vertical"),
    tf.keras.layers.RandomContrast(0.1),
])(inputs)

```

```

x = base_model(x, training=True)
x = GlobalAveragePooling2D()(x)
x = BatchNormalization()(x)
x = Dense(512, activation="swish")(x)
x = Dropout(0.6)(x)
output = Dense(5, activation="softmax")(x)

model = Model(inputs = inputs, outputs = output)
model.summary()

# am declarat x variabila de output a modelului, dupa l am facut intr un vector 2D, dupa l
am facut sa nu invete 30%

# (pentru stabilitate sa nu avem overfitting) dupa am conectat toate straturile, dupa am
setat 5 outputuri (de la 5 levels 0,1,2,3,4)

# pentru clasa

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=5e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=4,
min_lr=1e-7, verbose=1)

early_stop = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=7,
restore_best_weights=True)

checkpoint = tf.keras.callbacks.ModelCheckpoint(
    'best_diabetic_model.keras',
    monitor='val_loss',
    save_best_only=True,
    mode='min',
    verbose=1
)

history = model.fit(train_ds, epochs=35, validation_data=test_ds,
class_weight=class_weight_dict, callbacks=[reduce_lr, early_stop])

model.save("retina_test_320_vs4.h5")

acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(len(acc))

```



```

plt.figure(figsize=(16, 6))

plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy', color='blue', linewidth=2)
plt.plot(epochs_range, val_acc, label='Validation Accuracy', color='red', linewidth=2)
plt.title('Training and Validation Accuracy', fontsize=14)
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss', color='blue', linewidth=2)
plt.plot(epochs_range, val_loss, label='Validation Loss', color='red', linewidth=2)
plt.title('Training and Validation Loss', fontsize=14)
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
y_true = []
y_pred = []

for images, labels in test_ds:
    preds = model.predict(images, verbose=0)
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(preds, axis=1))

y_true = np.array(y_true)
y_pred = np.array(y_pred)

cm = confusion_matrix(y_true, y_pred)
class_names = ['No DR (0)', 'Mild (1)', 'Moderate (2)', 'Severe (3)', 'Proliferative (4)']

print("\n--- RAPORT DE CLASIFICARE ---")
print(classification_report(y_true, y_pred, target_names=class_names))

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names,
yticklabels=class_names)
plt.title('Matricea confuziilor', fontsize=14, pad=20)
plt.ylabel('Etichete Reale (True Labels)', fontsize=12)
plt.xlabel('Etichete Predise (Predicted Labels)', fontsize=12)
plt.tight_layout()
plt.show()

```

```
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout,
BatchNormalization
import os

os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
print("TensorFlow este configurat sa ruleze pe CPU pentru siguranta memoriei!")

print("Se reconstruieste arhitectura EfficientNetB0 pentru TF 2.7...")

try:
    base_model = tf.keras.applications.EfficientNetB0(include_top=False, weights=None,
input_shape=(320, 320, 3))
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = BatchNormalization()(x)
    x = Dropout(0.2)(x)
    outputs = Dense(5, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=outputs)
    print("Arhitectura a fost creata. Se incarca greutatile din .h5...")

    model.load_weights("retina_test_320_vs4.h5", by_name=True, skip_mismatch=True)
    print("Modelul a fost incarcat cu succes prin transfer de weights!")

except Exception as e:
    print(f"Eroare critica la incarcare: {e}")
    exit()

class_names = ['No DR (0)', 'Mild (1)', 'Moderate (2)', 'Severe (3)', 'Proliferative (4)']

cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

if not cap.isOpened():
    print("Eroare: Nu s-a putut deschide camera USB!")
    exit()

print("Sistemul ruleaza live la rezolutie optimizata (640x480). Apasa 'q' pentru oprire.")

while True:
    for _ in range(5):
        cap.grab()

    ret, frame = cap.retrieve()

    if not ret:
        print("Eroare: Nu s-a putut citi cadrul de la camera.")
        break

    display_frame = frame.copy()
```

```

img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
img_resized = cv2.resize(img_rgb, (320, 320))
img_float = img_resized.astype(np.float32)

blur = cv2.GaussianBlur(img_float, (11, 11), 0)

img_accentuare = (img_float - blur) * 4.0 + 128.0
img_accentuare = np.clip(img_accentuare, 0.0, 255.0) / 255.0

input_tensor = np.expand_dims(img_accentuare, axis=0)

preds = model(input_tensor, training=False).numpy()
clasa_predisa = np.argmax(preds[0])
probabilitate = preds[0][clasa_predisa] * 100

text = f"Diagnostic: {class_names[clasa_predisa]}"
cv2.rectangle(display_frame, (0, 420), (640, 480), (0, 0, 0), -1)
cv2.putText(display_frame, text, (15, 455), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0),
2)

cv2.imshow("Licenta - Real-Time DR Screening", display_frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```